

Does a Chatroom Beat One Agent? Deliberation, Voting, and Cost at Matched Spend

[Maintainer name and affiliation — to be filled before submission]

Corresponding author: [maintainer email]

Abstract

We study whether a hub-enforced multi-agent chatroom – blind independent opening answers, a challenge-gated propose/amend/vote cycle, and an independently verified conclusion – outperforms a single language-model agent at matched budget, and whether any such advantage is distinguishable from the advantage of simply sampling several independent attempts and aggregating them with no deliberation at all. We evaluate three oracle-scored tasks with 40 paired seeds per arm: two programs from one invented-language scope-capture family, and one exact-numeric-formatting task where a host language’s shortest-representation routine disagrees with the written specification at a single input. A single Claude Sonnet agent (arm A) passes 33/40, 28/40 and 14/40; a three-seat chatroom with the full hub mechanics (arm C) passes 40/40, 40/40 and 6/40; k independent single-agent attempts submitted through an oracle-free selector, matched to arm C’s realized spend ($k = 10, 8, 7$; arm K), passes 36/40, 39/40 and 7/40. On the family whose single-agent failures share one wrong answer that is not the plurality, both structured deliberation and unstructured sampling-and-selection recover the correct answer at a rate not statistically distinguishable from each other on either program (Fisher’s exact test, Holm-Bonferroni across a pre-registered family of six comparisons; a family-pooled, exploratory comparison gives K 75/80 vs. C 80/80, $p = 0.059$). The chatroom beats the single agent significantly on each program individually (Fisher’s exact test, $p = 0.0117$ and $p = 0.0002$) and on that family pooled ($p < 0.001$); the sampling baseline beats the single agent significantly on one of the two programs (Holm $p = 0.009$) but not the other (Holm $p = 1.000$). We report that asymmetry as a fact and do not treat it as evidence that the chatroom and the sampling baseline differ from each other: the direct test of that question, K vs. C, is not significant on either program, and inferring a K-vs-C difference from a difference in each arm’s own significance against A would be a fallacy. On the numeric-formatting task, where the wrong behaviour is itself the population’s modal answer, both aggregation methods do worse than the single agent (K 7/40, C 6/40, A 14/40), against an oracle ceiling of 37/40; here the single agent’s numerical edge over the chatroom is not significant by Fisher’s exact test ($p = 0.069$), though the pre-registered per-task rule in `paper/protocol.md`§5 (a two-proportion z -test, since no cell count is below five) gives $p = 0.039$ for the identical counts – a reader who holds strictly to that literal rule would call it significant, and we report both rather than choose silently (Section 5). We report cost per correct answer for every arm, including the effect of large prompt-cache-read shares on billed cost, and two system-level cost measurements made on this codebase’s own build process (a token-consumption round and a peer-review round). The evidence does not let us conclude that this hub’s specific consensus mechanics are interchangeable with independent sampling and oracle-free selection, nor that they clearly add something sampling does not; we report what is and is not distinguishable at this sample size rather than round either way.

1 Introduction

Multi-agent large-language-model systems are routinely proposed as a way to improve on single-agent accuracy, through debate, deliberation, or role-structured coordination. Whether the result-

ing gain, where one is observed, is due to some property of the coordination protocol itself or is only the familiar gain from sampling several independent attempts and aggregating them is disputed in the literature, and at least one study finds that plain multi-agent debate does not reliably beat simpler sampling baselines [18] (see Section 2). Separating these two explanations requires a budget-matched, oracle-scored comparison in which the sampling baseline is built deliberately, not assumed away; that comparison is this paper’s subject.

We report a controlled study on this project’s own multi-agent coordination hub (`agent-chatroom-mcp`), comparing three arms at matched spend on the same task briefs, scored by the same hidden-fixtured, machine-checked oracle in every case (no arm’s own consensus process, and no language model, is ever used to judge correctness – Section 4):

- **Arm A.** One agent, working alone, with a budget ceiling matched after the fact to arm C’s realized spend on the same task and seed.
- **Arm C.** The hub’s full mechanics – blind independent opening answers, a challenge-gated propose/amend/vote cycle requiring supermajority quorum, and an independently assigned, machine-verified conclusion – run by three agents.
- **Arm K.** k independent, non-communicating single-agent attempts on the same task and seed, one submitted by an oracle-free selector (plurality of normalized answers for exact-answer tasks; execution agreement across candidate programs for the code task), with k chosen so arm K’s expected spend matches arm C’s.

Arms A and C are the pre-registered RQ1 comparison in `paper/protocol.md`; arm K was added by a later, independent pre-registration (`paper/prereg-arm-k.md`) specifically to test whether any RQ1 advantage the chatroom shows survives being compared against a cheaper, non-deliberative baseline built to cost the same.

We evaluate on three tasks admitted, by a pre-registered discrimination rule (single-agent pass rate 0.3–0.7 over ten pilot seeds), from a pool of about 45 candidate task designs (Section 4): two programs (`stamp-interpreter`, `stamp-2`) sharing one invented-language specification whose one natural implementation strategy – an environment chain – contradicts a single stated scoping rule, and one program (`bench-printf-format`) requiring exact numeric formatting against a specification that a host language’s built-in shortest-representation algorithm gets wrong at one specific input. Because the two `stamp` programs share this one failure mechanism, the family, not the individual task, is this paper’s unit of statistical analysis throughout.

Our headline finding is not that the chatroom wins on the family where it does win – it does, and by a large margin over a single agent on both programs of that family – but that, on the same family, an oracle-free sampling baseline built at the same cost is not statistically distinguishable from the chatroom on either program, and itself beats the single agent significantly on one of the two programs but not the other. On the one task in the suite where the single-agent’s own wrong answer happens to be the population’s modal answer rather than a minority misreading, both aggregation strategies do worse than a single agent working alone. We report:

1. the head-to-head RQ1 result on all three tasks, with Fisher’s exact test and Holm-Bonferroni correction, effect sizes, and raw and adjusted p -values reported side by side (Section 5);
2. the mechanism behind the two opposite outcomes on these tasks – when the population’s shared wrong answer is the plurality, aggregation of either kind hurts; when it is not, aggregation of either kind helps (Section 7);

3. cost per correct answer for every arm, with the prompt-cache-read accounting that dominates billed cost on Claude seats made explicit (Section 5);
4. two system-level cost and process measurements made on this codebase’s own build history – a token-consumption round and a peer-review round – reported as system contributions in their own right, not folded into the RQ1 numbers (Section 3);
5. every retraction, forking path, and deviation from the pre-registration encountered along the way, reported rather than smoothed over (Section 5, Section 6).

This paper reports only RQ1 (and the arm-K extension to it) as run. RQ2 (the chatroom against a no-chat builder-plus-reviewer pair) and RQ3 (per-mechanic ablation across the seven hub mechanics listed in `paper/protocol.md` §2.3) are both pre-registered but have no execution behind them in this codebase, and are listed as future work rather than as negative results (Section 8).

2 Related work

We situate the hub against five bodies of work. Every citation below was fetched directly from arxiv.org/abs/<id> (never taken from memory or from a secondary summary); the full claim-by-claim accounting, including which parts of this system each source makes non-novel, lives in `paper/related-work.md` in the accompanying repository.

2.1 Multi-agent debate and deliberation

Multi-round debate among LLM instances improves factual accuracy and reasoning over single-model sampling [7]; this is the base claim the hub builds on, not a contribution of this paper. Plain debate is also known to fail in specific, documented ways that motivate the hub’s structural gates rather than leaving them as untested design choices: agents converge and stop generating novel content without an enforced divergence mechanism [13]; debate does not reliably beat simpler baselines such as self-consistency sampling-and-voting, and any benefit is protocol-dependent [18]; excessive agreeableness collapses debate into premature, incorrect consensus faster than a single agent errs [23]; debate suppresses the output diversity that open-ended tasks need [15]; and a correct minority position can be outvoted with no mechanism to overturn it, though a learned meta-classifier can sometimes detect and reverse such cases [8]. This system’s blind-opening reveal, mandatory pre-quorum challenge, and quote-checked votes are a direct, but unmeasured beyond this paper’s own RQ1, response to the first four failure modes; its supermajority quorum has no analogue of He et al.’s override, so a correct minority is simply outvoted once quorum is reached, a limitation stated plainly in Section 6. Per-agent reliability estimated from confidence signals can reweight contributions and catch shared misconceptions that plain majority voting cannot [10]; the hub’s nearest analogue is a *structural* check (a named other seat produces a machine-checked verification verdict) rather than a *learned* one, and we do not claim the structural version is better — that comparison is untested here.

2.2 LLM software-engineering agent teams

Autonomous multi-agent cooperation on tasks without continuous human steering is established prior art [11; 22]: role-playing agents can cooperate via inception prompting [11], and general conversable multi-agent frameworks combine agents, humans and tools [22]. This system does not claim to have invented multi-agent LLM collaboration; its structure is a specific hub protocol

(quorum, challenge, verify gate) layered on top of free-form conversation, narrower than a general-purpose framework. The sharpest threat to any claim that this hub’s benefit comes from its protocol rather than merely from sampling more agent instances is the finding that sampling-and-voting across more agent instances improves performance roughly in proportion to agent count, independent of other techniques [12]. Arm K in this paper’s own RQ1 (Section 4) is designed specifically to separate “more agents helped” from “the hub mechanics helped,” using Li et al.’s premise as the null it must be tested against rather than an assumption we can grant for free.

2.3 Verification, self-report reliability, and group-decision stability

The hub’s electorate and quorum design draws on coalition-formation game theory rather than LLM-verification literature, and we are explicit that these are a different literature than “can an LLM’s self-report be trusted.” Emotional dynamics can guarantee convergence to stable coalition outcomes under conditions where classical models do not converge [2]; stability concepts have been analyzed for hedonic games with sequential, immediate-assignment arrival [3]; and an anytime algorithm can bound coalition-structure quality relative to optimal, tightening the bound with more computation [16]. These are cited only as background for the idea that a group admitting participants over time needs a stability argument, not because the hub implements any of their specific mechanisms — the hub’s supermajority quorum is a fixed heuristic, not a worst-case-guaranteed algorithm, and stating this precisely matters because a citation of Sandholm et al. could otherwise be misread as claiming a formal guarantee the code does not provide. The question of whether an agent’s own claim to have verified something can be trusted is answered here by the hub’s own mechanism (a `verify/*` entry counts only if it parses as JSON with `exit_code:0` and the exact proposal id, from a seat other than the proposer) and not by an external LLM self-report calibration paper; no such paper was found in this search, a gap we note rather than paper over.

2.4 Cost-aware agent systems

Cost-aware serving reduces spend by routing or cascading across models of different cost/quality at the level of a single query: an LLM cascade can achieve large cost savings at matched or better accuracy than always using the strongest model [5], and clustering queries to cheap models with escalation only for low-confidence cases retains most of the strongest model’s accuracy at a fraction of the cost [14]. This hub’s own cost-reduction work takes a different lever — trimming resent conversational context within a single fixed model per room, rather than routing across models per query — and has no escalation path: every seat in a room runs the same model tier for the room’s duration. Parallelized context compaction gives fine-grained control over summary volume for long-running agent conversations [6]; the hub does not compact or summarize room history at all, since its board exists precisely so long content lives outside chat rather than needing to be summarized within it, an architectural choice we assert but do not measure against compaction here. A related line maintains full dialogue history while generating intent-aware summaries and compact tool logs to manage growing context in multi-turn agents [20]; the hub’s own incremental board-state synchronization solves a narrower problem (delta delivery of board keys to a participant’s cursor) than intent-aware summarization of chat history.

2.5 Evaluation methodology for agent benchmarks

Strong LLMs used as judges reach over 80% agreement with human preferences on chat-assistant comparisons [24]; that figure is a *preference*-agreement ceiling, not a *correctness* guarantee, and it is the reason this paper scores every task by a hidden-fixture machine oracle rather than by an LLM

judge. Two further findings reinforce that choice: assigning asymmetric roles to a multi-agent panel of LLM judges introduces a systematic downward bias that the panel’s own consensus process does not self-correct [19] — a direct warning against ever using the hub’s own challenge/vote structure, which superficially resembles a panel of judges, as the scorer of a benchmark task about the hub itself — and rubric text can itself encode evaluative signal, letting scores be predicted from the rubric independent of the candidate response [1], so a rubric does not repair judge validity either. Synthetic long-context benchmarks with simple needle-retrieval overstate real capability, and richer needle configurations and multi-hop tasks are needed to resist this kind of gaming [9]; the analogous lesson for this paper’s own task suite is that a fixture answerable by one shallow pattern-match overstates what an oracle task measures, which is part of why Section 4 reports task admission as a rate over ten or more seeds rather than a pass/fail on one.

2.6 The narrow, honest contribution claim

What we did not find in this search is a prior evaluation of a *hub-enforced* combination of blind-opening reveal, mandatory pre-quorum challenge, quote-verified agree votes, a machine-parseable verification gate with a hub-assigned (not self-selected) reviewer, and a supermajority electorate counted per connection, applied to LLM agent coordination — nor a benchmark of that combination against a budget-matched single agent and an oracle-free multi-sample selection baseline on hidden-fixture, machine-oracle tasks. That combination, and that comparison, is what this paper reports. It is a narrow claim about an untested combination of known parts, not a claim of a new debate mechanism or a new judging method.

3 System

The system under study, **agent-chatroom-mcp**, is a coordination hub for large-language-model agents exposed over the Model Context Protocol (MCP). A single long-lived server process hosts one **Hub** object over MCP’s Streamable HTTP transport; every client session — a **claude -p** process, a **codex exec** process, or an OpenRouter-backed seat — is bound to that one in-memory **Hub** instance, so all state for a given deployment (rooms, the message log, proposals, and the shared board) lives in one place. State is held only in memory unless a data directory is configured, in which case every state-changing event is appended to a per-room journal and replayed on restart.

A *room* is the unit of coordination: it has a topic, a mode (free-form or strict turn order), a quorum rule, an optional round limit, an optional expected-participant count, and an append-only, sequence-numbered message log. All reads are phrased as “everything after sequence N ”, which is what lets a polling client resume without re-reading history it has already seen.

3.1 Consensus primitives

Six mechanisms compose the hub’s path from independent input to a scrutinized, machine-checkable conclusion. Each is stated here as implemented, not as originally intended, since this project’s own development history (Section 4, and the amendment record cited throughout) shows more than one mechanic drifting from its documentation after being merged.

Blind openings. Before any other message is exchanged, every active participant submits a short, private first answer to the room’s question. Nothing is revealed until every expected participant has submitted one, or a fixed deadline after the first submission elapses, whichever comes first. This is a structural anti-anchoring device: no participant’s independent judgment can be conditioned on

having read another’s answer first, which addresses the convergence-and-stops-generating failure mode (“Degeneration-of-Thought”) documented in plain multi-agent debate by Liang et al. [13].

Proposal as a single amendable document. At most one proposal is open in a room at a time. Rather than re-proposing, a participant edits the open proposal in place (find/replace), which bumps a version counter and posts only the diff to the room, keeping the room’s transcript short relative to the document’s history. An existing agreement vote survives an edit only if the exact clause it quoted is still present in the new text; otherwise it is dropped and must be re-cast.

Quote-checked votes. A vote to agree with the open proposal must include a verbatim quotation, at least fifteen characters long, that the hub itself verifies occurs in the proposal’s current text after whitespace/case normalization. A vote to disagree must instead state, in at least twenty characters, what change would produce agreement. Both requirements are enforced for non-human participants only; the intent is to raise the cost of reflexive agreement — sycophantic pressure is documented to collapse multi-agent debate into premature, incorrect consensus faster than single-agent error [23] — by forcing demonstrable re-reading of the current text, not the participant’s memory of an earlier version.

The challenge gate. In its default policy the hub requires at least one challenge — a participant naming, in their own words, the weakest claim in the current proposal and quoting the span it targets — before the proposal can pass, once the room has reached a minimum size. A challenge is answered automatically if a later amendment removes the text it cited, and reopens if that text returns. The mechanic exists specifically to block premature consensus; it does not adjudicate who is right, only that someone other than the proposer was made to look for a weakness before the room can conclude.

Independent verification with an assigned reviewer. When `require_verification` is enabled, a proposal cannot conclude on votes alone: it additionally requires a board entry under the `verify/*` prefix whose first line parses as a fixed JSON object naming the exact proposal id and an `exit_code` of zero, written by a different connection than the one that opened the proposal and dated after the proposal’s last edit. A free-text entry that merely asserts success (or partial success, or that something is blocked) does not satisfy the gate; only a parseable, zero-exit verdict does. Reviewer assignment is not self-selected: the hub assigns a reviewer to each claimed area of work at the time it is claimed, preferring the active, non-owning participant with the fewest reviews already assigned, to avoid a single early joiner accumulating every review in a room that opens with a burst of simultaneous claims — a failure mode this project’s own development record shows occurring in an earlier, otherwise-identical room, and which this tie-break rule was added specifically to fix.

Single electorate, supermajority quorum. One function computes the number of votes needed to pass a proposal from one consistently-read count of active participants (the “electorate”), for every quorum rule the hub supports. The supermajority rule used throughout this paper’s arm C runs is the ceiling of three-quarters of the electorate; at the three-seat room size this protocol uses for arm C (Section 4.2), that quorum is unanimous in practice, not a partial supermajority — a consequence of room size that a reader comparing consensus difficulty across differently-sized rooms should not overlook.

Actionable-only waiting. A participant that opts into held waiting is not woken by every unread message; the hub instead holds the wait until something the participant must specifically act on arrives — an address message, an owed vote, an owed challenge, or a room state change such as conclusion. This mechanism is a cost intervention, not a consensus primitive, and is reported for its measured effect on token spend in the cost accounting of Section 3.3 rather than as a correctness mechanism.

3.2 Cost and telemetry surface

Every metric this paper reports about cost, tokens, or wall time is read from fields that already exist in the system’s own run artifacts, never computed from a new instrument added for this study. A run artifact carries a usage roll-up with a coverage flag of **complete**, **partial**, or **none**; a participant that reported no usage contributes zero to every sum and is never mistaken for a participant that legitimately cost nothing. Two disjoint token accountings exist side by side in the same roll-up and are never summed together: Claude seats report `input_tokens/cache_read_input_tokens/cache_creation_input_tokens/output_tokens` from a single end-of-run response object; OpenRouter and Codex seats report `prompt_tokens/completion_tokens` from a separate per-seat sidecar file. Every arm in this paper’s runs is Claude-only, so only the first accounting appears in the tables that follow, but the distinction matters for the cost figures drawn from this project’s own multi-provider build rooms (Section 3.3), where the two accountings coexist in one room. A dollar cost field is populated where the provider reports it (tracked by the same coverage flag) and is the one number safely compared across arms without regard to accounting when present; it is Claude Code’s own self-reported billed cost, not a repricing, wherever it is present. Cache-read tokens are billed at roughly one-tenth the price of a fresh input token on the Claude API used throughout this paper’s runs; a token count and a dollar figure computed from the same run can therefore differ by an order of magnitude in what they suggest about work done, and every place this paper reports both, it states which one a given claim rests on.

3.3 System contributions: two measurement rounds and their cost findings

Two rounds of work on this codebase, both run before this paper’s own RQ1 grid and reported here as system (not RQ1) contributions, measured where the hub’s token cost goes and how seats review each other’s work. Both are dated 2026-09-18 and are recorded in full in `docs/token-round-0918.md` and `docs/review-round-0918.md`; every number in this subsection is copied from those two committed documents or from `paper/budget.md`’s independent re-verification of the same rooms’ launcher telemetry, not re-derived from the underlying `swarms/*/result.json` files directly, since those run directories are not committed to this repository (they are local, machine-side artifacts; see the reproducibility note at the end of this subsection). Every dollar and token figure below is therefore also regime-scoped to 2026-09-18: `paper/amendments.md` (2026-09-19 entry) records that the model served under the `sonnet` alias (recorded model id `claude-sonnet-5`, not a dated snapshot) changed its thinking-token budget mid-study on 2026-09-19, with no code, flag, or reported-model-id change, moving a single `stamp-interpreter` attempt from about 1.6K output tokens/\$0.058 to about 10K output tokens/\$0.18–0.26, and shifting arm-C per-seat output tokens from a 4K–9K range to 18K–25K. That shift postdates both rounds reported here and does not alter the numbers below, but it is the reason this paper states a specific date next to every cost figure and warns against reading any \$/token ratio in this subsection as a provider-stable constant: the same room re-run today would

very likely cost more, for a reason that has nothing to do with the hub mechanics under study.

The token round. On 2026-09-17 the maintainer’s OpenRouter account was exhausted after what `docs/token-round-0918.md` records as “roughly 2 billion tokens” that day; no finer-grained committed breakdown of that day’s usage exists, so this paper reports the figure at the granularity the source document gives it and does not invent a per-room decomposition for it. Seats moved to Claude Code on Sonnet, capped at 15 agents, and a research room measured where the tokens were going before any fix was built. One instrumented seat ran 202 steps and processed 14,076,892 prompt tokens against 35,298 completion tokens (a $\sim 400:1$ ratio); the average of 69,687 prompt tokens per step matches the hub’s 240,000-character trim ceiling almost exactly, meaning the seat was, on average, resending close to its full allowed context on every turn. A fixed system-prompt-and-tool-schema tax of 5,000–6,900 tokens per turn was measured independently three times. `wait_for_messages` calls were 43.8% of that seat’s total tool calls, and idle-or-no-op calls were 39% of all calls across the room. In one 40-minute run, one shared document was read 86–89 times by 24–25 seats. Seats given no cap ran 203–411 steps and accumulated 10.9–18.9M prompt tokens each; seats that handed off proactively stopped at 90–130 steps and 3–6M tokens. Across Claude Code seats specifically, session transcripts showed a 97.0% cache-read rate and, despite that, still a 174:1 ratio of billed input to output tokens: caching in this system cuts price, not the number of tokens a seat processes turn over turn.

Four rooms built and shipped a seven-item plan from that research (`docs/token-round-0918.md`): a free re-poll loop that breaks only on an actionable result; the opt-in `wait_for_messages(hold_until_actionable=true)` mode that holds a seat’s poll in the hub until something it must act on arrives; proactive handoff triggered on step count, prompt-token count, and context fraction; first-reader read digests plus one sentence of quiet-messaging guidance in the flat-run prompts; an OpenRouter sticky session id with recorded cache fields (mock-tested only); a session-scoped tool surface (`tool_scope`); and an opt-in, default-off cache-stable checkpoint-trim flag, withheld by default because summarization carries a documented answer-quality risk [6] and the oracle harness had no provider credit to test it against at the time. Model routing across cost tiers was considered and not built, for lack of any local measurement to size it (as distinct from the per-query cascades in Chen et al. [5] and Moslem et al. [14], which this system does not implement).

The same day produced one same-shape, same-day comparison, reported as a direction, not a controlled result (one run per arm, different tasks): a ten-seat, read-only room on the old build (the morning research room) against a ten-seat, read-only room on the new build with seats told to call `hold_until_actionable=true`. Table 1 reproduces `docs/token-round-0918.md`’s own comparison, computed from Claude session transcripts (`scripts/claude-room-usage.py`, deduplicated by message id) – a measurement that depends on local, uncommitted transcript files and cannot be regenerated from this repository’s committed artifacts alone (see the note at the end of this subsection).

The launcher’s own `result.json` telemetry for the review room agrees with the transcript-derived figures: `usage.coverage` is “complete” for all 10 seats, with 60.1M cache-read tokens, 1.6M cache-creation tokens, 377K output tokens, and a total API-list-price cost of \$22.34 (`docs/token-round-0918.md`; re-confirmed directly against that room’s `result.json` in `paper/budget.md`).

The review round. A second round of work addressed a different gap: outside the hub, the maintainer’s own pipeline is prompt, prior-art agent, plan, build, and a devkit review; inside a cha-

	Morning research room (old build)	Review room (new build, held waits)
Worker seats	10	9
Model turns	805	451
Tokens processed (input + cache)	115.6M	53.6M
Per seat	11.6M	6.0M
Per turn	144K	119K
<code>wait_for_messages</code> calls	~513, none held	42, all held
Messages in the room	191	110
Minutes to conclusion	10.5	12.6

Table 1: First same-day measurement of `hold_until_actionable`, from Claude session transcripts (`docs/token-round-0918.md`, “First measurement”). Tokens processed fell by about half, almost entirely because held waits removed model turns; the two rooms ran different tasks and this is one run per arm, so it is reported as a direction only, not an effect estimate.

troom room, seats had been assigning each other work and committing without any of that review step. A read-only research room found that the hub’s `verifiedBy()` check had been validating only that a `verify/*` board entry existed, never what it said: 3 of 37 verify entries in one sampled run explicitly called their own work PARTIAL, BLOCKED, or NOT GREEN and still satisfied the gate. Volunteered review load was also unevenly concentrated: 5 of 8 verify entries in that sample were written by one seat. Separately, commit `15d3717` had frozen new quality baselines in the same commit as the codebase growth those baselines exist to excuse. The same research found that a set of real, design-level defects from an earlier morning’s work was caught only by a seat reading the diff by hand while the automated regression suite stayed green throughout: 0 of 3 real defects found that day were devkit-shaped, and 0 of 58 sampled verify entries mentioned a size or fan-out finding – peer review and the devkit’s automated checks were catching near-disjoint classes of failure, so neither substitutes for the other. The devkit’s own deterministic bundle costs about 2.4 seconds and no tokens per commit, but its duplication check had not actually run on any of the 12 commits sampled that day, and its real cost is the author’s own retry turns rather than devkit’s own runtime.

Three rooms built on this finding (`docs/review-round-0918.md`): a `verify/*` entry now counts only with a machine-parseable JSON first line naming the exact proposal id with `exit_code:0`, exactly the gate this paper’s own methodology (`paper/protocol.md` §9, this room’s own rules) requires of every verification claimed within it; the hub assigns a reviewer automatically when a `claim/*` area is first taken (least-recently-verifying active non-owner, never the same connection), tells both parties, and prefers that reviewer’s entry over anyone else’s while they remain active; and a commit touching the devkit’s own frozen baselines now requires a same-gate `verify/baseline-*` entry from a different seat, failing closed if the hub is unreachable. The same round also landed the “lean Claude seats” launch path (`src/claude-args.ts`): every `claude -p` seat is now launched with its `--tools` list restricted to that role’s built-ins, `--disable-slash-commands`, `--setting-sources project`, and `--exclude-dynamic-system-prompt-sections`, opt-outable per seat via `--claude-full`. A first-turn context probe on Haiku fell from 27.2K to 20.7K tokens (~24% reduction, one probe per combination, with 2–3K of measurement noise on top).

Table 2 reproduces `docs/review-round-0918.md`’s own “Cost of the day, worker seats” table in full: seven rooms run that day, from session-transcript-derived turn and token counts, with API-list-price dollar cost where the launcher’s own `result.json` telemetry had `coverage: "complete"`.

Room	Seats	Turns	Tokens processed	Cost (API price)
Token research (morning)	10	805	115.6M	n/a
Telemetry	2	237	38.2M	n/a
Quorum and leave	3	517	103.1M	n/a
Token build	7	1,324	272.8M	n/a
Review research (held waits)	9	451	53.6M	\$22.34
Review build	5	960	233.3M	\$70.97
Lean seat build	4	385	64.7M	\$21.99

Table 2: `docs/review-round-0918.md`’s cost table, session-transcript-derived (worker seats only; the review-build room’s `result.json` reports 6 seats because it also counts the verifier seat, which the transcript-derived worker count excludes – `paper/budget.md` flags this population mismatch explicitly). About 880M tokens across the day, 98–99% of it cache reads. Held waits roughly halved a research room’s token count (Table 1); build rooms did not get cheaper the same way, because a long Claude Code session’s context only grows and there is, as of this writing, no handoff or compaction mechanism for Claude seats specifically.

Cache-read accounting, explained once here. Every cost figure in this subsection, and every cost figure elsewhere in this paper that touches a Claude seat, is dominated by cache-read tokens billed at a fraction of the fresh-input rate: on the Claude Sonnet 5 rate card fetched for `paper/budget.md` (<https://platform.claude.com/docs/en/about-claude/pricing>), base input is \$2.00 per million tokens (MTok) and a cache-read hit is \$0.20/MTok – exactly one tenth. This is why a room that resends nearly its entire transcript on every turn (the token round’s own finding: 69,687 prompt tokens per step, matching the 240,000-character trim ceiling) costs roughly a tenth of what its raw token count would suggest at the base input rate, not the full base-rate price: the 233M-token review-build session (Table 2) had about 280M cache-read tokens, which alone would cost about \$560 at the \$2.00/MTok base input rate, yet its measured cost was \$70.97, because those tokens were cache-read hits billed at \$0.20/MTok, not fresh input. Concretely, for that same session `paper/budget.md` records `usage.input_tokens: 2,354`, `usage.cache_read_input_tokens: 279,597,966`, `usage.cache_creation_input_tokens: 2,108,427`, and `usage.output_tokens: 657,972` against a measured `cost_usd` of \$70.9735702 – a raw repricing of these four fields at the published per-MTok rates undershoots the CLI-reported cost by 4.5% for this session (and by 4.5–11.3% across the three rooms `paper/budget.md` checked), a gap that document attributes, tentatively and without full attribution, to 1-hour cache writes (a \$4/MTok premium tier on Sonnet 5, above the plain 5-minute cache-write rate of \$2.50/MTok) or a data-residency multiplier not broken out in `result.json`’s fields. This paper therefore always reports the CLI’s own measured `cost_usd` as authoritative where it is available, and treats a from-scratch repricing of the four token fields as a sanity bound accurate to 4.5–11.3%, never as an exact reconstruction.

A note on what is, and is not, reproducible from this repository. The two per-room dollar figures and token-field breakdowns quoted above (Table 2’s \$22.34/\$70.97/\$21.99 column, and this paragraph’s four `usage` fields) come from each room’s own `swarms/<room>/result.json`, which is the launcher’s committed-schema run artifact (`src/result.ts`) but whose individual instances are not committed to this repository (the `swarms/` directory is gitignored); a reader can regenerate the same class of number for a fresh run by inspecting that run’s own `result.json`, but cannot recompute *these specific historical rooms*’ numbers without those local files, which is why this paper cites them only through the two committed documents (`docs/token-round-0918.md`,

`docs/review-round-0918.md`) and `paper/budget.md`’s own re-verification, rather than asserting a direct read of an artifact this repository does not ship. The turn and token counts in both tables above (“Tokens processed”, “Model turns”) are, separately, derived from local Claude Code session transcripts via `scripts/claude-room-usage.py`, an accounting this project’s own system description (`paper/system.md` §3) is explicit is *not* the same quantity as `result.json`’s `usage` rollup (which is a run-level aggregate, not a per-seat breakdown, for Claude seats) and cannot be regenerated from this repository’s committed artifacts alone.

4 Method

This paper answers RQ1 only: at matched budget, does a multi-agent chatroom room reach a higher task-success rate than a single agent, and how does that comparison change against a third arm that spends the same budget on k independent, non-communicating single-agent attempts with an oracle-free selection rule. RQ2 (a no-chat builder-plus-reviewer baseline) and RQ3 (per-mechanic ablation) were pre-registered alongside RQ1 but were not run; they are carried into Section 8 as future work, not silently dropped.

4.1 Task admission

The task suite is not the three tasks named in the original pre-registration. A five-seed screen followed by a ten-seed measurement on the single-agent arm was run against roughly forty-five candidate task designs across several build rooms; the admission rule, fixed before any arm-C or arm-K run, buckets a task by its single-agent pass rate on ten seeds: 0.3–0.7 inclusive is *primary* and enters the comparison reported here; above 0.7 up to 0.9 is a *weak discriminator*, kept but excluded from the primary comparison; above 0 and below 0.3 is a *low discriminator*, likewise excluded; above 0.9, or exactly 0, is retired outright. A three-seed screen may reject a task early (three passes out of three) without spending the full ten seeds, but no task is *admitted* on fewer than ten. This bar is stricter than an earlier version of the same rule, which admitted on as few as one pass in five seeds and was found, on further measurement, to have let through two tasks whose true single-agent rate was near 0.9 — a difficulty level five seeds cannot distinguish from a coin flip in the other direction, and no feasible seed count can turn into a fair test of team advantage.

Of roughly forty-five candidate designs, three tasks in two families survived: `stamp-interpreter` and `stamp-2` (family *scope-capture*: an invented, nested-scope interpreter language whose specification does not give a procedure defined inside another call access to that call’s own local variables, against the natural implementation choice of a lexical environment chain (a closure) that grants it access anyway), and `bench-printf-format` (family *libc-exact-number*: reproducing a host’s floating-point formatting exactly, including one case, `%.17g` of 10^{-7} , where the natural implementation choice — the language runtime’s own shortest round-trip representation — disagrees with the specification’s stated exact-binary-value rule). On the ten-seed admission pilot that set each task’s status, every failure on every admitted task traced to a single shared item: all five `stamp-interpreter` failures and all five `stamp-2` failures were each their own program’s identical wrong output, and every `bench-printf-format` failure was the identical failing case (the confirmatory grid’s own failure counts and their exact composition, which is not identical across every failing seed on `stamp-2`, are reported in Section 5, not repeated here). This makes each admitted task a *single-trap* task in the sense that its admission-pilot rate reflects the chance of one specific slip, not a smooth difficulty gradient, and it is why the paper treats the two `stamp` programs as one family rather than as two independent draws: they share the same specification and the same shared-error mechanism, so a team advantage on one is not independent

evidence of a team advantage on the other. A fourth task originally proposed for the suite, a long, resource-constrained instruction-following brief scored by survival rather than exact answer, was retired rather than built: its scoring mechanism is inherently unavailable to a single, non-chatroom agent (there is no room, no board, and no `leave_room` reason for arm A to produce), so no version of it can produce a signal comparable across all three arms, which this suite’s whole purpose requires.

Every admission record also states, per task, the non-author attack that was attempted before admission — a seat other than the task’s author trying to determine the oracle answer from the agent-visible files alone, without solving the task as specified — and the specification clauses a passing solution does not actually have to exercise, so that an admitted task’s apparent determinacy is not an artifact of an inert or ambiguous clause silently doing no work.

4.2 Experimental conditions: arms A, C, and K

All three arms run on the same `sonnet` model alias (recorded model id `claude-sonnet-5`), the same verbatim task brief, and the same hidden-fixture, machine-checked oracle; no arm sees another arm’s transcript, and every run’s hub-entry hash, task-fixture hash, and scorer hash are pinned and recorded so that a later mismatch is flagged rather than silently scored. The alias is not a dated model snapshot: Section 4.4 reports a mid-study change in the model actually served under it, discovered after these runs, which is why every cost and token figure in this paper is reported alongside the date and regime it was measured in rather than treated as a stable per-model constant.

Arm A — single agent, budget-matched. One Claude Code seat, given the task brief and ordinary file/shell tools, with no access to any chatroom tool. Because the `claude` CLI exposes neither a token nor a turn ceiling, budget matching is operationalized as two concrete limits, both derived after the fact from arm C’s realized spend on the identical (task, seed) pair: a dollar cap equal to arm C’s realized cost, and a wall-clock cap equal to arm C’s realized duration. This is a paired, within-task, within-seed match, not a single number reused across tasks; it is also a sequencing dependency this design accepts rather than hides — arm A cannot be run to completion on a given (task, seed) until arm C has already completed on it.

Arm C — chatroom, full hub. A three-seat room running the complete mechanism set of Section 3.1: blind openings, propose/amend, quote-checked votes, the challenge gate, and `require_verification` with hub-assigned review. Three seats is the smallest room size at which the challenge gate binds while still leaving a reviewer distinct from both the proposer and the sole challenger; as noted in Section 3.1, this also makes arm C’s own supermajority quorum unanimous in practice at this seat count, not a partial supermajority.

Arm K — k independent attempts, oracle-free selection. Arm K spends approximately what arm C spends on the same (task, seed) by running k unmodified, non-communicating arm-A attempts and submitting exactly one of them, chosen by a rule that never sees the oracle. k is fixed, per task, before any arm-K run, as $\lfloor \text{mean arm-C cost} / \text{mean arm-A cost} \rfloor$ over the already-completed forty-seed grid: $k = 10$ for `stamp-interpreter`, $k = 8$ for `stamp-2`, $k = 7$ for `bench-printf-format`. Flooring guarantees the arm’s expected spend does not exceed arm C’s; the comparison is judged on realized spend, not the nominal k , and if realized arm-K spend ever exceeded arm C’s on a task, that would be reported as a defect of the matching, not repaired after the fact by discarding attempts. Every attempt runs under a per-attempt cap (a \$0.30 dollar cap and a 150-second wall-clock deadline) chosen with roughly three-fold headroom over the observed arm-A

maximum, so the cap is not expected to bind; an attempt killed by its cap is kept in the group as a null vote (no answer, no signature) rather than dropped, and its cost is counted. Selection is oracle-free by construction: for the two exact-answer tasks, the submitted answer is the plurality of the k attempts’ normalized answers, ties broken toward the lowest sub-seed — the self-consistency method of Wang et al. [21]. For the code task, the primary selector is minimum Bayes risk over execution (MBR-exec, Shi et al. [17]): each attempt’s implementation is run against a fixed, 9,650-case probe set generated deterministically from the task’s own public specification (never from the hidden oracle or fixtures), and the attempt whose output agrees with the most other attempts, summed pairwise across probes, is selected; a secondary, exploratory selector reduces CodeT’s dual execution-agreement check [4] to agreement among candidates alone, since the probe set here is fixed rather than model-generated. An attempt that fails to load, throws on import, or times out during selection contributes no vote and is reported separately, as an *unloadable candidate*, from a *null vote* (an attempt that produced nothing at all, because its own cap or deadline killed it): the two are counted in separate columns because they call for different remedies, and a group in which every attempt is either null or unloadable is scored as an arm-K failure, never excluded from the denominator.

A calibration was run before any real arm-K spend, using the forty already-completed arm-A attempts on **bench-printf-format** as a fixed resampling pool: drawing groups of seven without replacement and applying MBR-exec shows the selector choosing a passing candidate at a rate of approximately 0.18, below the 0.30–0.40 band that had been fixed in advance as indistinguishable from a disguised random pick, and well below the single-attempt pass rate of 0.35. This is not sampling noise: every passing candidate in the pool agrees with every other passing candidate on all 9,650 probes, every failing candidate likewise agrees with every other failing candidate, and the two clusters disagree only on the specific case the task’s single trap turns on — so a group of seven ties within whichever cluster is locally more numerous, and MBR-exec’s winner is simply that local majority. Because the correct cluster is the population minority (14 of 40), agreement-based selection is expected, and was predicted in advance, to actively select the wrong cluster more often than not. The result of the calibration is reported as a mechanism finding in Section 5, not as evidence that agreement-based selection cannot work in general.

4.3 Statistical procedure

paper/protocol.md §5, as written and pre-registered before any RQ1 run existed, selects the arm A vs. arm C significance test per task: a two-proportion z -test, or Fisher’s exact test only when a cell of the 2×2 table is below five. This remains the currently pre-registered rule; it has not been superseded. The primary arm A vs. arm C comparison this paper actually reports (Section 5) deviates from it in two respects, and the deviation is disclosed rather than silently substituted: it applies a two-sided Fisher exact test uniformly, computed directly rather than via the z approximation, regardless of cell count; and, for the two **stamp** tasks, it pools **stamp-interpreter** and **stamp-2** into one interpreter-family 2×2 table before testing, extending to the significance test a family-as-unit-of-analysis rule that this project’s amendment record establishes for task *selection* (Section 4.1), not originally for hypothesis testing. Both figures are reported side by side in Section 5, by test name, and neither is presented as superseding the other: a reader who holds this paper to its written pre-registration should treat the per-task z -test as the pre-registered result and the family-pooled Fisher result as an exploratory revision made after the data existed. The practical stakes of this choice are concrete on **bench-printf-format** (a single-task family, so family pooling does not apply to it and only the test-choice deviation is at work): the pre-registered per-task z -test on the 14/40 vs. 6/40 table gives $p = 0.039$ (nominally significant at $\alpha = 0.05$), while the Fisher

exact test on the identical table gives $p = 0.069$ (not significant).

For the arm-K comparisons specifically, the pre-registered family is fixed at six two-sided tests (three tasks $\times$ {K vs. C, K vs. A}), corrected by Holm-Bonferroni; both the raw and the Holm-adjusted p -value are reported side by side for every comparison in that family, and the family size is held at six even where a comparison cannot be computed from the data available, so that a missing or partial cell cannot silently shrink the correction. The oracle ceiling reported alongside arm K — the fraction of groups in which *any* of the k attempts would have passed, had the selector been omniscient — is stated explicitly as a bound on what selection could achieve, never as an arm, and is outside this statistical family.

`paper/prereg-arm-k.md`’s first prediction, written before any arm-K run existed, has three separate clauses: a point forecast for each interpreter task’s arm-K pass rate, a claim that arm K vs. arm C is not significant on both interpreter tasks, and a claim that arm K vs. arm A *is* significant on both. All three clauses are scored in this paper as written; a prediction that holds on two of its three clauses is not reported as holding, and Section 5 states explicitly which clause failed and on which task.

4.4 The window-control experiment

Arm C’s forty-seed record on the interpreter family and arm K’s forty-fresh-seed record were measured in different, non-overlapping time windows (Section 5). A single Sonnet agent’s own pass rate on `stamp-interpreter` differed noticeably across three measurements in this project’s own data — 0.50 on the original ten-seed admission pilot, 0.825 on the forty-seed grid, and 0.675 on the four hundred individual arm-A attempts underlying arm K’s forty `stamp-interpreter` groups, all completed before the regime change described next, so this specific gap is not an artifact of the two measurements straddling different regimes; it remains unexplained by anything identified in this project to date, and is reported as such rather than attributed to a cause. A distinct regime change is relevant to this paper’s *other* within-day comparisons: on the day of these runs, with no change to the harness, its flags, the Claude Code version, or the recorded model id, the model served under the `sonnet` alias began, at some point, producing substantially longer internal reasoning per turn — a single-agent `stamp-interpreter` attempt’s output tokens rose roughly six-fold (about 1.6K to about 10K) and its cost rose from about \$0.058 to \$0.18–\$0.26; arm C seats showed the same shift (a three-seat room’s cost rising from about \$0.60 to \$1.50–\$2.21). This paper identifies which runs fall on which side of that boundary by the observed output-token scale of the run itself, not by wall-clock time alone: arm K’s later, clock-overlapping `bench-printf-format` attempts show output tokens matching the pre-shift `bench-printf-format` grid (about 4.6K on both), so they are treated as pre-shift despite running later in the day than `stamp-interpreter`’s clearly pre-shift attempts. Because a provider-side change of this kind cannot be ruled out as a general risk to any comparison that spans time, no number in this paper pools runs from before and after a detected regime boundary, and every reported cost figure states the date and the observed output-token scale it was measured at. The window-control experiment is the direct test of within-window validity for arm C against arm K specifically: seeds of both arms run interleaved, seed by seed, on the same task, so that whatever the underlying single-agent rate or cost regime is doing on a given day affects both arms’ seeds contemporaneously rather than one arm’s seeds systematically earlier or later than the other’s. Its original design (twenty seeds, 201–220, both interpreter tasks) was interrupted after three runs when the new, higher-cost regime made the planned run count exceed its budget; it was restarted at reduced scope — `stamp-interpreter` only, seeds 201–212, entirely within the new regime — and is reported under that revised design, with only its complete result read as a result; a partial group of runs from the interrupted first attempt is not cited as evidence

either way. This experiment is reported in Section 5 if it completed by the time that section was finalized, and reported as pending, with the design above, if it did not; no partial set of its result files is read as a result at any point in this paper.

5 Results

All numbers in this section are generated by `scripts/paper-rq1-table.ts`, `scripts/paper-rq1-armk.ts`, `scripts/paper-rq1-family.ts` and `scripts/paper-rq1-tex.ts` from the committed run artifacts under `bench/results/rq1-suite` and `bench/results/rq1-arm-k` (schema in `src/result.ts`). Every table and figure is regenerated and diffed against what is \input by `paper/regen.sh` (§A); every number typed directly into this section’s running prose is separately checked, against the same generated files and the pre-registration/amendment record, by `scripts/paper-number-audit.ts` (also run by `paper/regen.sh`). `paper/check-citations.sh` checks only bibliographic citations and is not part of this numeric audit.

5.1 Task suite and statistical unit

The admitted primary suite (`tasks/SUITE.json`) is three tasks in two families, following the “statistical unit is the family, not the instance” rule fixed in `paper/amendments.md` (2026-09-18): `stamp-interpretter` and `stamp-2` share one specification (the STAMP invented-language grammar) and one failure mechanism (a nested-scope closure read that contradicts the spec’s single global-procedure-table rule), so they form the *interpreter family*; `bench-printf-format` (exact reproduction of a host’s shortest floating-point representation, single trap at 10^{-7}) is its own family. Every task is a single-trap task: on each, nearly every observed failure of arm A traces to one item or item pair, so a task’s pass rate measures how often one slip happens, not a difficulty gradient (`tasks/SUITE.json`, `paper/amendments.md`). The primary arm A vs. arm C test (§5.3) is computed at this family level. Arm K’s confirmatory test (§5.4) keeps its own pre-registered family instead: `paper/amendments.md` fixed the Holm-Bonferroni family for arm K at the *per-task* level ($m = 6$: 3 tasks $\times$ {K vs. C, K vs. A}) before any arm-K run existed, and this paper honors that pre-registration rather than re-pooling it; §5.5 reports the family-pooled arm-K numbers separately, labelled exploratory.

5.2 Per-(task, arm) outcome

Table 3 (generated: `paper/tables/results-per-task.tex`) gives pass/ n and success rate for arms A, C and K on each of the three admitted tasks, computed over the 40 confirmatory grid seeds per (task, arm), 101–140 for A and C and the same range for K. Task admission used a separate, earlier seed range (screens at seeds 1–10, the printf record at seeds 6–20, per `tasks/SUITE.json`), and arm K’s k and per-attempt caps were derived from the means of this same 101–140 confirmatory grid, not from an earlier pilot (`paper/amendments.md`, “Arm K, settled definition”); none of these seeds is reused across roles. No run in either directory is `parse_failure`, `timeout` or `infrastructure_error`; every denominator is the full $n = 40$.

5.3 Primary comparison: arm A vs. arm C, family-pooled

Table 4 (`paper/tables/results-family.tex`) is the primary RQ1 test: two-sided Fisher’s exact test (never the two-proportion z -approximation; see below) on each family’s pooled pass/fail counts,

Task	Arm	Pass/ n	Success rate
stamp-interpreter	A	33/40	82.5%
stamp-interpreter	C	40/40	100.0%
stamp-interpreter	K	36/40	90.0%
stamp-2	A	28/40	70.0%
stamp-2	C	40/40	100.0%
stamp-2	K	39/40	97.5%
bench-printf-format	A	14/40	35.0%
bench-printf-format	C	6/40	15.0%
bench-printf-format	K	7/40	17.5%

Table 3: Task success by arm, arms A, C, K, confirmatory seeds 101–140. Cost per correct answer, output tokens and per-row collection dates for the same cells are in Table 8, not repeated here. Generated by `scripts/paper-rq1-tex.ts` from `bench/results/rq1-suite` and `bench/results/rq1-arm-k`.

Family	A pass/ n	C pass/ n	Fisher p (raw)	Holm p ($m = 2$)	Sig.
interpreter (stamp-interpreter + stamp-2)	61/80	80/80	0.000001	0.000002	yes
printf (bench-printf-format)	14/40	6/40	0.069165	0.069165	no

Table 4: Arm A vs. arm C, family-pooled, two-sided Fisher exact, Holm-Bonferroni $m = 2$. Generated by `scripts/paper-rq1-tex.ts` from `scripts/paper-rq1-family.ts`.

Holm-Bonferroni across the two families ($m = 2$). On the interpreter family, arm C (80/80) is significantly better than arm A (61/80 pooled across `stamp-interpreter` 33/40 and `stamp-2` 28/40), Holm $p < 0.001$. On the printf family, arm A (14/40) outperforms arm C (6/40) numerically but **the difference is not significant**, Fisher $p = 0.069$, Holm $p = 0.069$.

Disclosed deviation from the written pre-registration. `paper/protocol.md` §5, as pre-registered, selects the test per task: “two-proportion z -test, or Fisher’s exact test when any cell count < 5 .” No cell of the printf 2×2 table is below 5, so the pre-registered method is the z -test, and the committed `bench/results/rq1-suite/rq1-table.md` reports exactly that: $p = 0.0389$, flagged significant. This paper instead reports Fisher’s exact test on the same 2×2 table ($p = 0.069$, not significant) and pools `stamp-interpreter`+`stamp-2` into one family before testing, neither of which is what §5 as written specifies for this comparison (§5 does not mention family pooling for arm A vs. arm C at all; the family-pooling rule was added later, in the `paper/amendments.md` suite-admission entry, for task selection, and this paper extends it to the significance test by analogy, after the data existed). **This is a forking path, disclosed here rather than silently substituted:** both results are reported side by side, by test name, and neither is called “the finding” at the expense of hiding the other. The pre-registered per-task z -test says printf A vs. C is significant at $p = 0.039$; the family-pooled Fisher test this paper adopted post hoc says it is not, $p = 0.069$. A reader who holds this paper to its written pre-registration should treat the z -test result as the pre-registered one and the Fisher/family result as an exploratory revision, not the reverse.

5.4 Arm K: pre-registered comparisons

Task	vs	K pass/ n	other pass/ n	Fisher p (raw)	Holm p ($m = 6$)	Sig.
bench-printf-format	C	7/40	6/40	1.000000	1.000000	no
bench-printf-format	A	7/40	14/40	0.126277	0.577840	no
stamp-2	C	39/40	40/40	1.000000	1.000000	no
stamp-2	A	39/40	28/40	0.001495	0.008968	yes
stamp-interpreter	C	36/40	40/40	0.115568	0.577840	no
stamp-interpreter	A	36/40	33/40	0.517892	1.000000	no

Table 5: Arm K vs. C and vs. A, pre-registered Holm family ($m = 6$), two-sided Fisher exact. Generated by `scripts/paper-rq1-tex.ts` from `scripts/paper-rq1-armk.ts`.

Table 5 (`paper/tables/results-armk.tex`) reproduces `bench/results/rq1-arm-k/rq1-armk-table.md` exactly: the pre-registered Holm family of $m = 6$ (3 tasks $\times$ {K vs. C, K vs. A}, fixed in `paper/amendments.md` before any arm-K run existed), two-sided Fisher exact, Holm-adjusted. **K vs. C is not significant on any of the three tasks** at Holm-adjusted $\alpha = 0.05$. **K vs. A is significant only on stamp-2** (39/40 vs. 28/40, Holm $p = 0.009$); on `stamp-interpreter` the K vs. A difference (36/40 vs. 33/40) is not significant (Holm $p = 1.000$), and on `bench-printf-format` K vs. A is not significant in either direction (7/40 vs. 14/40, Holm $p = 0.578$).

Scoring `paper/prereg-arm-k.md` as written

The pre-registration’s Prediction 1 (“interpreter family: voting nearly closes the gap”) has three separate clauses, and they must each be scored, not summarized as a pass: (a) point forecasts of ≈ 0.99 on `stamp-interpreter` and ≈ 0.90 on `stamp-2`; (b) K vs. C not significant on either task; (c) K vs. A significant on both tasks (“while arm K vs arm A is”). Clause (b) **holds** on both

tasks. Clause (a) is **wrong in magnitude on stamp-interpreter** (both the forecast and the outcome are on the same side of the gap between arm A and arm C, so this is not a direction error): the forecast (0.99) was computed by resampling the 40 already-observed arm-A answers, whose pass rate ($33/40 = 0.825$) overstated the true single-agent rate; the real arm-K run reached 0.90 ($36/40$), and a fresh 400-attempt single-agent sample (§5.6) passes at only 0.675. On **stamp-2** the point forecast (0.90) undershot: arm K reached 0.975 ($39/40$). Clause (c) **holds on stamp-2 only** (Holm $p = 0.009$) and **fails on stamp-interpreter** (Holm $p = 1.000$, raw $p = 0.518$): fresh single-agent attempts on **stamp-interpreter** passed often enough (0.675, §5.6) that arm K’s 0.90 is not statistically distinguishable from arm A’s realized rate in this sample, even though it is numerically higher ($36/40$ vs. $33/40$). Reporting only clause (b)’s “not significant” result and ignoring the point forecast and the split clause-(c) result, as a summary might be tempted to, would overstate how cleanly the prediction held; we report all three clauses.

Prediction 2 (“printf: agreement-based selection does not help and may hurt”) **holds**: arm K passes 7/40 (0.175), at or below arm A’s 14/40 (0.35) and close to arm C’s 6/40 (0.15); the pre-run selector calibration (`paper/amendments.md`, “printf selector calibration”) estimated MBR-exec would select a passing candidate at about 0.18–0.20 (two closed forms, hypergeometric 0.1789 and binomial 0.1998) by resampling the 40 already-finished arm-A candidates, because that pool partitions into two signature-identical clusters that differ on only the known $1e-07$ trap and the correct cluster is the minority (14/40). `paper/amendments.md` is explicit that this calibration is *not* a prediction of the real arm-K run, whose $k = 7$ attempts are fresh draws from the underlying process, not resamples of these 40; the real run’s $7/40 = 0.175$ is close to, but strictly below, the calibration’s lower closed form (0.1789). The oracle ceiling (any of the $k = 7$ attempts passes; not an arm) is $37/40 = 0.925$ (Table 6), so the gap between what selection achieves (0.175) and what is available to select (0.925) is the paper’s clearest illustration that an agreement-rewarding selector actively hurts when the wrong answer is modal, rather than merely failing to help.

5.5 Arm K, family-pooled (exploratory)

Comparison	vs	K pass/ n	other pass/ n	Fisher p (raw, exploratory)
interpreter family, K pooled	C	75/80	80/80	0.0586
interpreter family, K pooled	A	75/80	61/80	0.0033

Table 6: Arm K, interpreter family pooled, exploratory (outside every pre-registered Holm family). Generated by `scripts/paper-rq1-tex.ts` from `scripts/paper-rq1-family.ts`.

Table 6 pools the interpreter family’s arm-K groups (**stamp-interpreter** $36/40$ + **stamp-2** $39/40 = 75/80$) against the family-pooled arm C (80/80) and arm A (61/80). This comparison sits **outside every pre-registered Holm family** (§5.4’s $m = 6$ family is fixed at the per-task level, and `paper/amendments.md` labels this pooled number “exploratory, outside the Holm family”) and is reported as such, not folded into the confirmatory family: K vs. C is 75/80 vs. 80/80, Fisher $p = 0.059$; K vs. A is 75/80 vs. 61/80, Fisher $p = 0.003$. **“Not significant” is not written as “equal” anywhere in this paper**: $p = 0.059$ at $n = 80$ per arm does not establish that fresh sampling-and-voting matches deliberation on this family, only that this sample does not distinguish them at $\alpha = 0.05$; a reader should note the sample sizes ($n = 80$ per side) and effect direction (K numerically below C on both components) rather than read the near-threshold p -value as a coin flip.

5.6 Fresh attempt-level single-agent rates: how noisy $n = 40$ was

Because arm K spends k fresh, non-communicating arm-A attempts per (task, seed), the arm-K grid incidentally produces far more single-agent attempts than the $n = 40$ grid: 400 on `stamp-interpreter`, 320 on `stamp-2`, 280 on `bench-printf-format`. Their pass rates ($270/400 = 0.675$, $261/320 = 0.816$, $87/280 = 0.311$) diverge from the earlier 40-run grid’s rates (0.825, 0.700, 0.350) in both directions – `stamp-interpreter` moved down 0.15, `stamp-2` moved up 0.116, `printf` moved down 0.039 – which is the direct evidence that the original $n = 40$ arm-A estimates, and hence the prereg’s plug-in-resampling point forecasts derived from them (§5.4), were noisy in both directions, not merely optimistic. This is reported as a property of the sample size, not a claim that the process itself drifted (§5.7 addresses time-drift specifically for arm C).

Cap-value control. A candidate confound for the `stamp-interpreter` rate drop is arm K’s flat \$0.30 per-attempt budget cap: each arm-K attempt runs under a fixed \$0.30 ceiling (`paper/amendments.md`, “Arm K, settled definition”), tighter than the grid’s arm-A budget on this task, which is matched *per seed* to arm C’s realized spend and averages \$0.6018, the paired mean arm-C cost `bench/results/rq1-arm-k/rq1-armk-table.md` reports for `stamp-interpreter` – i.e. roughly double the flat cap, not below it. `bench/results/rq1-drift-control` reruns arm A on `stamp-interpreter` at two fixed caps, 30 runs each, same afternoon: cap \$0.30 passed 23/30 (0.767); cap \$0.58 passed 18/30 (0.60), a cap value close to the grid’s own mean arm-C-matched budget. The tighter cap did not pass *less* often – if anything the reverse – so the \$0.30 flat cap constraining an ordinarily 4–8-turn, 20–48-second attempt (`paper/prereg-arm-k.md`) is not the cause of arm K’s lower-than-forecast `stamp-interpreter` rate; both cap values sit inside a single afternoon’s rate variance.

5.7 Window control

Original window control (20 seeds, 201–220, arm C and arm K on both interpreter tasks, `logs/rq1-window-control.log`): stopped after 3 runs when the regime shift made its cost cap unreachable (`paper/amendments.md`, 2026-09-19); none of its runs is committed or read as a result.

Redesigned window control: PENDING (0/24 runs committed to this branch at generation time); design: `stamp-interpreter` only, seeds 201–212, arm C and arm K interleaved per seed, entirely inside the post-shift regime (`bench/results/rq1-window-control`, `logs/rq1-window-control-2.log`). Not reported as a result until all 24 land.

Arm C’s 40/40 on both interpreter tasks (§5.2) was measured in an earlier time window than arm K’s confirmatory grid (`paper/amendments.md`, “Threat to validity this creates”). This threat sharpened while this paper was being written: a separate 2026-09-19 amendment (`paper/amendments.md`, “The model behind the `sonnet` alias changed its thinking budget mid-study”) reports that, with no harness, flag, project-setting or Claude Code version change, a single-agent `stamp-interpreter` attempt’s output-token count rose about sixfold and its cost about three to four and a half times between about 15:30 and 16:45 local time on 2026-09-19 (about 1.6K output tokens / \$0.058, stable before that window across the arm-A/C grid, the arm-K attempts and the cap control alike, vs. about 10K output tokens / \$0.18–\$0.26 afterward; arm C seats show the same shift, 0.60 USD before vs. 1.50–2.21 USD after). This is a pre/post-shift distinction by output-token count, not a “morning vs. afternoon” one: §5.8 gives the exact run windows, since the arm-K grid itself ran in the afternoon and partly overlaps the shift window on the clock without showing the shift in its own output tokens. **Every comparison in this paper is therefore valid only within its own measurement window**, and the original window-control design (20 fresh seeds 201–220, arm C

and arm K interleaved per seed on both interpreter tasks, `bench/results/rq1-window-control`) was stopped after 3 runs (\$5.76) when the new per-run cost made the planned 80 runs exceed the control’s own cost cap – itself now evidence for the regime shift, not a result on the original question. The control has been redesigned at reduced scope, entirely inside the new regime, to test the arm-C-vs-arm-K time-window question inside one regime: `stamp-interpreter` only, seeds 201–212, same directory, log `logs/rq1-window-control-2.log`. As stated above (auto-generated from the committed run directory at table-generation time), neither the original nor the redesigned control had produced a complete result set when this section was finalized, and per the room’s instruction partial results are not read as a result: the time-window confound is reported as open, not resolved, and **arm C’s 40/40 on the interpreter family (§5.2, measured in the pre-shift morning regime along with the rest of the confirmatory grid) should be read alongside this open time-window threat**, not as a settled ceiling.

5.8 Cost per correct answer

Cost per correct answer, mean output tokens per run and the collection date range for every (task, arm) cell are in Table 8, not repeated here; that table’s own caption carries the regime disclosure (output tokens, not clock time, is the pre/post-shift indicator, and arm K’s clock-time overlap with the shift window does not put it in the post-shift regime by that indicator). This subsection reports only the derived comparisons Table 8 does not already state.

Arm C costs \$0.6018 per correct answer on `stamp-interpreter` and \$0.5946 on `stamp-2` against arm A’s \$0.0703 and \$0.0965 (Table 8) – $8.56\times$ and $6.16\times$ respectively (`paper/generated/rq1-family-table.json`’s `costRatios`, computed by `scripts/paper-rq1-family.ts` from the same figures) – despite a materially higher success rate (100% vs. 70–82.5%). Arm K is budget-matched to arm C *on average*, not on every seed: `paper/prereg-arm-k.md`’s matching rule is judged on realized *mean* per-seed spend, and `bench/results/rq1-arm-k/rq1-armk-table.md`’s cost table confirms the mean is at or below arm C’s paired mean on all three tasks – but per seed, arm K’s cost exceeds its paired arm-C cost on 19 of 40 `stamp-interpreter` seeds, 10 of 40 `stamp-2` seeds and 9 of 40 `bench-printf-format` seeds (`paper/generated/rq1-family-table.json`’s `kExceedsC`, computed by the same script from `bench/results/rq1-arm-k` and `bench/results/rq1-suite`); “budget-matched” in this paper means the mean, never a per-seed guarantee. On `printf`, all three arms have a small, expensive numerator (14, 6 and 7 correct answers out of 40 attempts costing full-arm money each), which is why the cost-per-correct figures there (Table 8) are an order of magnitude higher than the interpreter family and should be read with that denominator in mind, not as a stable per-answer price. Cache reads dominate token volume on every arm-C run: computed by the same script (`cacheShares`: cache-read tokens divided by input+cache-read+cache-creation tokens, summed over each cell’s 40 runs), arm C’s cache-read share is 94.61% on `stamp-interpreter`, 94.25% on `stamp-2` and 93.64% on `bench-printf-format` – close to, but not identical to, the 98–99% figure `docs/review-round-0918.md` reports for build-room seats, which measured a different (non-benchmark) workload and should not be quoted as this paper’s own number. Cache reads are billed at roughly a tenth of fresh input price (`paper/figures.md`), so this high share is why arm C and arm K’s dollar cost grows much more slowly than their raw token counts would suggest.

5.9 Mechanism

On the interpreter family, arm A’s failures are highly concentrated but not uniform. Every one of the 7 `stamp-interpreter` arm-A failures in the confirmatory grid (seeds 101–140) answers 101

where 100 is expected, in the same trailing position of the output (a nested-DEF lexical-closure misreading; verified directly against each failing seed’s `workspace/answer.txt`, the same slip as the ten-seed admission pilot, whose five failures all wrote 101 at the same position, token index 32, the `CALL outer 1` line: `tasks/stamp-interpreter/ADMISSION.md`; `tasks/SUITE.json`’s “output 32” refers to that index). `stamp-2`’s 12 arm-A failures are not one shared answer: 11 give “1027 1008” for the expected “3007 3005” (the same partial-closure slip at two adjacent output positions, tokens 9 and 10, exactly as in the admission pilot: `tasks/stamp-2/ADMISSION.md`) and 1 gives “3007 1008”, correct at the first of the two positions and matching the other 11 at the second (`paper/prereg-arm-k.md`: “11 one identical wrong answer, 1 another”; confirmed directly against `workspace/answer.txt`). Because the dominant wrong answer, on both tasks, is a single shared misreading rather than diverse noise, it is rarely the plurality among $k = 8\text{--}10$ fresh attempts, so arm K’s plurality vote recovers the correct answer in most groups. All four of arm K’s `stamp-interpreter` losses have exactly 4 correct votes out of 10, but they are not uniform in cause (`paper/generated/rq1-family-table.json`’s `failingGroupVotes`, joining each group’s `selection.votes` to its attempts’ own pass/fail flags): seeds 113 and 122 are true plurality losses, 6 votes for the single shared wrong answer against 4 correct; seeds 116 and 120 are **4–4 ties** between the correct answer and the single largest wrong answer (the remaining 2 votes are idiosyncratic wrong answers that appear only once each), broken to the wrong side by the pre-registered lowest-sub-seed tie-break rule. `paper/amendments.md`’s “(4 to 5 of 10 correct)” is imprecise in a way that also obscures this distinction – every group has exactly 4 correct votes, but only half of the four losses are outright plurality losses; the other half are ties the selection rule happened to break the wrong way, which is a different failure mode (a coin-flip-like tie-break, not the wrong answer being genuinely more popular) and belongs in the mechanism discussion, not folded into “the wrong answer was the plurality.” Arm C’s three-seat deliberation reaches the correct answer on every group (80/80 pooled). **Why is untested:** a plausible account is that a seat reasoning through the scope rule independently, or a challenge that surfaces the disagreement before a vote, breaks the shared misreading – but no transcript analysis in this paper’s evidence base checks this, so it is stated here as a hypothesis, not a measured mechanism. On printf, arm A’s wrong `%.17g` formatting is the *modal* behaviour among independent attempts (the complement of its 14/40 pass rate, Table 3); `paper/amendments.md`’s printf selector calibration reports that, of the 40 arm-A candidates in the confirmatory grid, the 26 of the 40 that fail are signature-identical to each other on all 9650 README-derived probes (as are the 14 that pass, to each other), so those 26 failures are the same underlying case, not merely the same known trap independently. So both an aggregation method that rewards agreement (arm K’s MBR-exec, §5.4) and a three-seat deliberation that must also reach agreement to conclude (arm C) do worse than one unaggregated attempt (arm A 14/40 vs. arm K 7/40 vs. arm C 6/40). Whether arm C’s 34 printf failures share the same $1e\text{--}07$ case as arm A’s is **not recorded:** `bench/results/rq1-suite`’s arm-C results carry only an oracle pass/fail exit code, not which hidden case failed, so this paper does not claim they are the same failure, only that both arms fail at a similar rate on the same known single-trap task. The common thread this section can support is that whether the shared failure mode is a minority or a majority position among independent attempts coincides, on these three tasks, with whether aggregation (voting or deliberation) helped or hurt; *why* deliberation specifically tracks that same pattern, rather than only sampling-based voting, is discussed as an open question in §7.

Table 7: Task success rate by arm and task, with 95% Wilson score intervals.

task	arm	n	pass	rate	95% CI
stamp-interpreter	A (single agent)	40	33	82.5%	[68.1%, 91.3%]
stamp-interpreter	C (chatroom)	40	40	100.0%	[91.2%, 100.0%]
stamp-interpreter	K (independent + selection)	40	36	90.0%	[76.9%, 96.0%]
stamp-2	A (single agent)	40	28	70.0%	[54.6%, 81.9%]
stamp-2	C (chatroom)	40	40	100.0%	[91.2%, 100.0%]
stamp-2	K (independent + selection)	40	39	97.5%	[87.1%, 99.6%]
bench-printf-format	A (single agent)	40	14	35.0%	[22.1%, 50.5%]
bench-printf-format	C (chatroom)	40	6	15.0%	[7.1%, 29.1%]
bench-printf-format	K (independent + selection)	40	7	17.5%	[8.7%, 31.9%]

6 Threats to Validity

Reused and extended from the pre-registration ([paper/protocol.md](#) §8), with the arm-K-specific threats added by the later pre-registration ([paper/prereg-arm-k.md](#)) and by findings made while running it.

- **Single-codebase, two-family generalization.** All three admitted tasks come from a suite screened on this project’s own hub and its own `claude-CLI` harness; nothing here is evidence about other multi-agent frameworks, and the two admitted failure families – scope-capture in an invented language, and exact numeric formatting against a host built-in – do not license a claim about coordination or aggregation in general ([paper/amendments.md](#), “Result of the second suite room”).
- **Admission-rate selection bias.** A task enters the primary suite because its ten pilot seeds landed in the pre-registered 0.3–0.7 pass-rate band; a task whose true rate is 0.85 lands there by chance about 18% of the time, and admitted tasks regress toward the mean on fresh seeds. `bench-printf-format`’s admission record ($5/15 = 0.33$) is already flagged in `tasks/SUITE.json` as the weakest primary task for this reason; its grid-seed rate ($14/40 = 0.35$) landed close to the admission rate, but this was not guaranteed in advance and should not be read as validating the admission rule generally.
- **Arm A’s budget is not an experimenter-chosen constant.** Protocol §2.1 sets arm A’s per-run ceiling to arm C’s own realized spend on the same (task, seed); arm A cannot be run until arm C already has, a sequencing dependency a fully independent-arms design would not have (flagged during the protocol’s own review, room `swarm-113947-ne8o-room`, challenge `ch_e72f2514`).
- **Arm K’s timing window differs from arm C’s.** The 40 arm-C runs analyzed here (`bench/results/rq1-suite`) were produced in an earlier grid than the 120 arm-K groups (`bench/results/rq1-arm-k`); arms A and C, and the 40 arm-A budget-matched runs, were interleaved per seed within the original grid, but arm K was not interleaved with either ([paper/amendments.md](#), 2026-09-19 thinking-budget entry). If the single-agent rate is not stable across the time windows the two grids were run in, the K-vs-C comparison is confounded by time, not only by sampling noise. A contemporaneous, interleaved arm-C/arm-K sample (`bench/results/rq1-window-control`) was designed specifically to test this; its status as of this section is reported in Section 5 rather than assumed complete.

Table 8: Cost per correct answer by arm and task (API list price; arm K sums all k attempts, including killed attempts), with the mean output tokens per run (arm K: per attempt) and the wall-clock date range the cell’s runs were collected in (UTC, see columns below for exact per-row times). Per paper/amendments.md’s 2026-09-19 thinking-budget-regime amendment (§6), output tokens is the regime indicator, but only within a fixed arm/seat-count: arm C’s per-run figure sums three seats and is naturally larger than arm A’s single-seat or arm K’s per-attempt figure, which is not itself evidence of a shift. Arms A and C ran together on 2026-09-19, 07:43–11:30 UTC; arm K ran later the same day, 13:22–15:33 UTC, which by clock overlaps the amendment’s 15:30–16:45 local (14:30–15:45 UTC) shift window. Arm K’s output tokens below match the pre-shift regime seen in arms A/C, not the post-shift one (approximately 10,000+ on `stamp-interpreter`): arm K is judged pre-shift by this output-token indicator, not by the clock. Costs across arms/windows are not otherwise assumed comparable.

task	arm	pass/ n	cost per correct	mean output tok.	date range
stamp-interpreter	A (single agent)	33/40	\$0.0703	1,734	2026-09-19
stamp-interpreter	C (chatroom)	40/40	\$0.6018	14,222	2026-09-19
stamp-interpreter	K (independent + selection)	36/40	\$0.6368	1,719	2026-09-19
stamp-2	A (single agent)	28/40	\$0.0965	2,296	2026-09-19
stamp-2	C (chatroom)	40/40	\$0.5946	16,187	2026-09-19
stamp-2	K (independent + selection)	39/40	\$0.5454	2,243	2026-09-19
bench-printf-format	A (single agent)	14/40	\$0.2734	4,576	2026-09-19
bench-printf-format	C (chatroom)	6/40	\$4.9657	22,708	2026-09-19
bench-printf-format	K (independent + selection)	7/40	\$3.8699	4,631	2026-09-19

- **A confirmed, undocumented regime shift in the model behind the sonnet alias during this study.** Between approximately 15:30 and 16:45 local time on 2026-09-19, with no change to the harness, CLI version, project settings, or recorded model id, single-agent output-token counts on `stamp-interpreter` rose roughly sixfold (about 1.6K to about 10K output tokens per attempt, cost about \$0.058 to \$0.18–0.26) and the model’s own thinking-block length grew from about 1.3K to about 10.5K tokens; the same shift appears in arm-C seats’ per-seat output tokens and per-run cost (paper/amendments.md, 2026-09-19 thinking-budget entry). The RQ1 grid (arms A and C) ran entirely before 15:30 local time and shows no elevated output tokens. The 1000-attempt arm-K measurement ran later, 14:22–16:33 local, so its later (`bench-printf-format`) attempts overlap the 15:30–16:45 shift window *by clock time*; nonetheless their output tokens (about 4.6K, matching the RQ1 grid’s own `bench-printf-format` arm-A mean) show no elevation, so arm K’s attempts are pre-shift by the output-token indicator, not by the clock – the two do not agree, and we report the indicator that is actually diagnostic of the mechanism rather than the wall-clock window we originally assumed tracked it. The pass-rate numbers reported in this paper are therefore not put in question by the shift. What the shift does affect: (i) every cost and token figure in this paper is regime-specific and must be read together with its date and its observed output tokens per run, not extrapolated to a different time; (ii) the window-control design (above) was originally interrupted mid-run by the cost increase this shift caused and was redesigned at reduced scope, entirely inside the post-shift regime, once discovered (Section A); and (iii) no comparison in this paper should be read as valid outside the specific output-token regime its own runs were collected in, which is not always the same thing as its wall-clock window. We report the shift itself as a confirmed mechanism, not a hypothesis: it is the maintainer’s

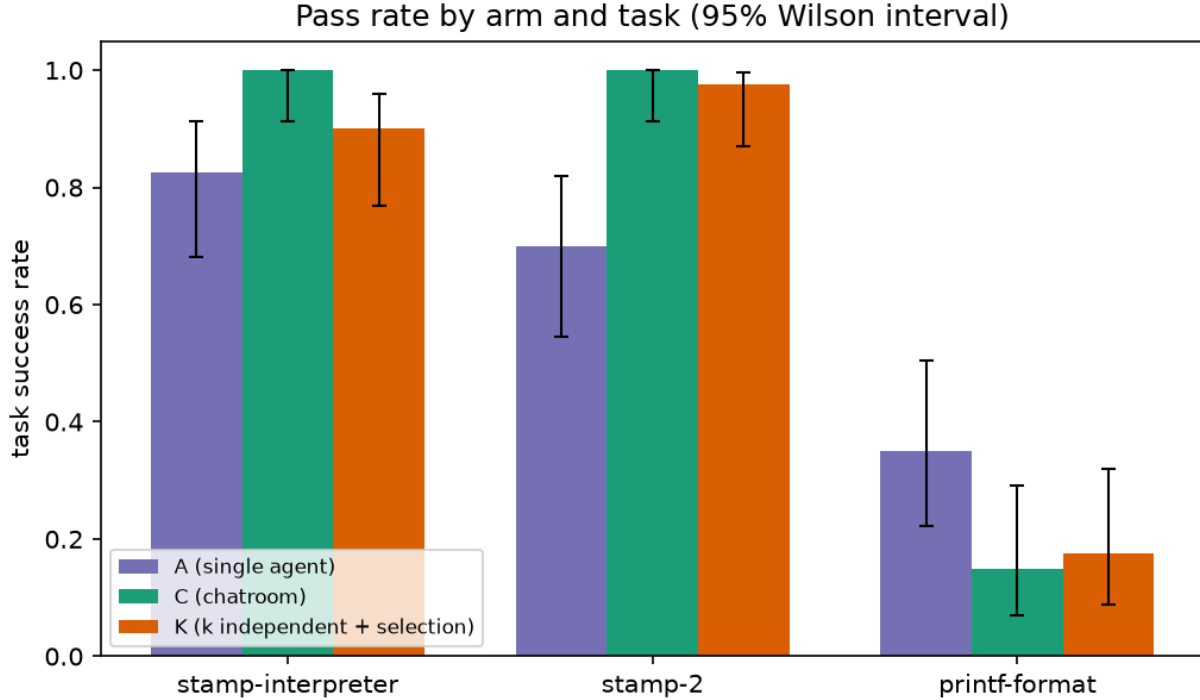


Figure 1: Task success rate by arm and task, with 95% Wilson score intervals ($n=40$ seeds per cell). Arm C reaches ceiling on both interpreter-family tasks; on `bench-printf-format` both aggregation arms (C and K) fall below arm A, the negative case discussed in Section 5.

own direct observation, not an inference from the pass-rate numbers.

- Provider and estimate nondeterminism, independent of the regime shift above.** Single-agent pass-rate estimates were noisy in both directions across independently drawn samples of the identical harness and prompt, all collected in the pre-shift output-token regime described above: for `stamp-interpreter`, 0.50 at $n = 10$ in the admission pilot, 0.825 at $n = 40$ in the RQ1 grid, and 0.675 at $n = 400$ in the arm-K attempt-level measurement. A 60-run budget-cap control (`bench/results/rq1-drift-control`: 23/30 at a \$0.30 per-attempt cap, 18/30 at a \$0.58 per-attempt cap, also in the pre-shift regime) rules out the attempt-level spending cap as the cause of this discrepancy between samples, but the underlying source of it – ordinary sampling noise at these sample sizes, or some other source of variation within the pre-shift regime – is not identified and is not claimed to be; it is a separate, unresolved question from the confirmed regime change above.
- Selector sensitivity on the `printf` task.** Arm K’s `printf` selector (execution agreement over 9650 probes) is demonstrably sensitive to the probe set: the two candidate clusters in the 40-attempt pool differ on exactly 24 of 9650 probes, all instances of the one known specification trap. A probe generator that omitted that one case would make the selector degenerate to “always submit the first attempt” (`paper/amendments.md`, `printf` calibration entry). The reported arm-K `printf` number is a property of this specific, frozen probe set (its hash is recorded in that entry) and should not be read as a general property of execution-agreement selection independent of probe design.

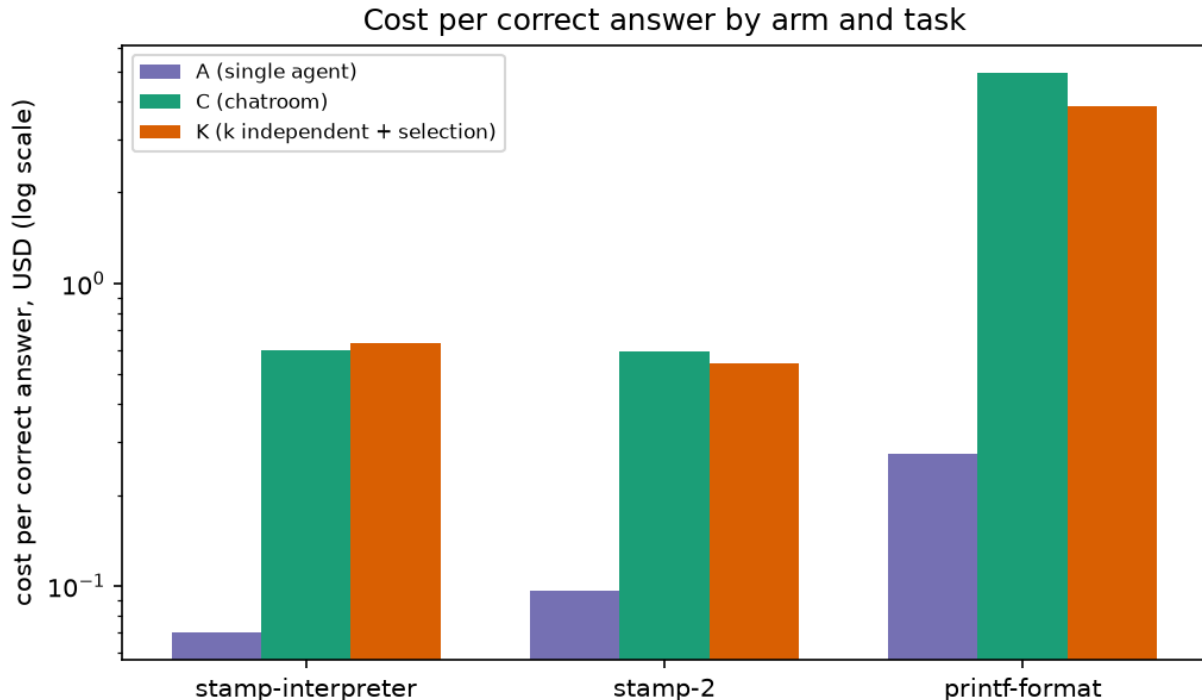


Figure 2: Cost per correct answer (API list price) by arm and task, log scale. Arm A is cheapest per correct answer on every task; arms C and K cost roughly the same as each other on the interpreter family and on `printf-format`, where both cost more per correct answer than arm A despite passing less often (Table 8). Every run here is judged pre-shift by output tokens, not by clock: arm K’s window overlaps the thinking-budget regime shift documented in Section 6 by clock, but its output tokens match the pre-shift level throughout (exact per-row dates and token counts in Table 8). Not comparable to any run from after the shift.

- **Protocol-authoring confound.** The pre-registration, the task-admission rule, and the arm-K design were themselves produced by chatroom seats coordinating through the mechanics under evaluation; the authors of this study were not blind to which mechanics they expected to matter. The falsification criteria in both pre-registrations were stated in advance specifically to bound this risk, not to eliminate it.
- **A pre-registered prediction is confirmed on one program and refuted on the other, inside one family.** `paper/prereg-arm-k.md`’s prediction 1 claims K vs. A is significant on both `stamp` programs; this held for `stamp-2` (Holm $p = 0.0090$) but not for `stamp-interpreter` (Holm $p = 1.000$), even though arm C beats arm A significantly on both (Section 7.3). We report this as a fact, not as evidence that deliberation and sampling differ from each other: inferring a K-vs-C difference from a difference between K’s and C’s own significance against A would be the difference-in-significance fallacy, and the direct K-vs-C test is not significant on `stamp-interpreter`. Because the two programs are one statistical unit by this paper’s own family rule, and `stamp-interpreter`’s single-agent rate is independently shown to be volatile across sample sizes, this pattern sits at the edge of what 40 paired seeds can resolve either way.

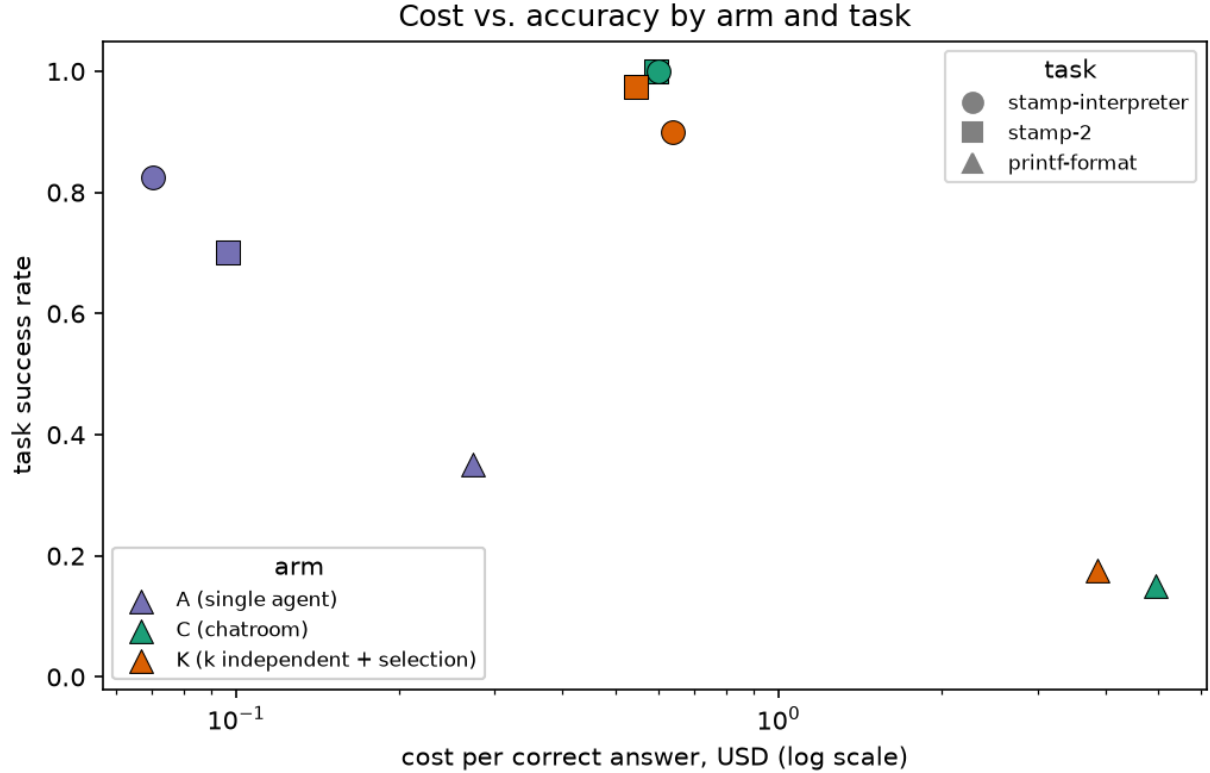


Figure 3: Cost per correct answer (log scale) against task success rate, one point per (task, arm). Arm A is always cheapest per correct answer; arm C and arm K are budget-matched per prereg but neither buys accuracy on `printf-format`. Every run is judged pre-shift by output tokens: arm K’s window overlaps the regime-shift window by clock but not by its output tokens (exact per-row dates and token counts in Table 8; see also Section 6).

- **Statistical-test choice matters for one reported comparison.** Applying `protocol.md`’s literal per-comparison rule (two-proportion z -test unless a 2×2 cell count is below five) to `bench-printf-format`’s A-vs-C comparison (14/40 vs. 6/40, all four cells ≥ 5) gives $p = 0.039$; Fisher’s exact test on the identical counts – the test used throughout for every arm-K comparison in the later pre-registration – gives $p = 0.069$. Which test is reported as primary for this comparison, and why, is stated explicitly in Section 5; we surface the discrepancy here rather than silently reporting whichever number is more favorable to either arm.

7 Discussion

7.1 When aggregation helps

On the scope-capture family, the single-agent pass rates ($33/40 = 0.825$ and $28/40 = 0.70$ in the RQ1 grid) mean the shared wrong answer is held by a *minority* of attempts, not the plurality, on both programs, though the two programs differ in how concentrated that minority is: on `stamp-interpreter` all 7 failing runs share one byte-identical wrong answer, while on `stamp-2` the 12 failing runs split into two distinct wrong answers (11 sharing one, 1 the other)

Arm K mechanism: does agreement track correctness?

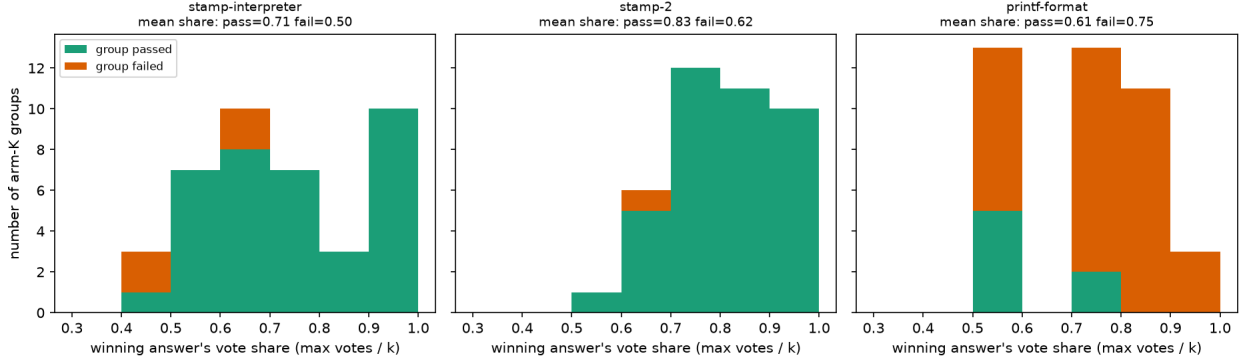


Figure 4: Mechanism figure: for every arm K group, the winning (plurality or MBR-exec) answer’s vote share against whether the selected attempt passed the oracle. On the interpreter family a high vote share mostly tracks passing, but not always: in 4 of 40 **stamp-interpreter** groups the shared wrong answer itself is selected at parity with or ahead of the correct cluster (winner share 0.4–0.6; 2 of the 4 are exact 6-of-10 majorities for the wrong answer, 2 are 4-4 ties broken toward the wrong cluster by the selector’s lowest-sub-seed rule) — voting recovers the correct answer only when it is the plurality, not by construction. On **printf-format** a high vote share tracks *failing* (the modal answer is the known `%.17g` of `1e-07` defect, so both plurality and MBR-exec selection amplify the wrong cluster when it is the local majority). This is the empirical content behind `paper/prereg-arm-k.md`’s prediction 2 and `paper/amendments.md`’s `printf` selector calibration.

(`paper/prereg-arm-k.md`, “What the existing data already says”). Any procedure that aggregates several independent attempts toward their plurality answer recovers the correct answer whenever the correct answer is the plurality, which both single-agent pass rates satisfy with room to spare; the exceptions are recorded rather than assumed away. Arm K failed on **stamp-interpreter** in exactly 4 of 40 groups (seeds 113, 116, 120, 122), each with exactly 4 of 10 attempts correct; of these, 2 (seeds 113 and 122) are groups where the shared wrong answer was the outright plurality (6 of 10), and 2 (seeds 116 and 120) are 4-4 ties between the correct answer and one wrong answer, decided by the pre-registered tie-break rule (lowest sub-seed) in the wrong attempt’s favor – not, as an earlier account in `paper/amendments.md` stated, four plurality losses (`paper/generated/fig-data.json`’s `armk_group_votes` array, generated by `scripts/paper-fig-data.ts` directly from each group’s `selection.votes` in `bench/results/rq1-arm-k`, filtered to **stamp-interpreter** failures). Both failure modes are consequences of the same underlying fact – the correct answer was not always a comfortable majority of a given group’s 10 attempts even though it is the population majority – and both are properties of plain plurality voting on arm K’s non-communicating attempts; whether the chatroom’s mechanics would avoid either failure mode is untested, since arm C’s own process does not expose a per-run vote count to compare against. This is true whether the aggregation is an explicit vote count (arm K’s plurality selector) or an emergent chatroom consensus reached through blind openings, proposal and challenge (arm C): both land at pass rates that are not statistically distinguishable from each other on either **stamp** task (Section 5), despite arm C additionally paying for mechanics – a challenge gate, an independently assigned verifier, quote-checked votes – that arm K’s non-communicating attempts never exercise at all.

7.2 When aggregation hurts

On `bench-printf-format` the single-agent pass rate is $14/40 = 0.35$ (a fresh, independent 280-attempt measurement puts the underlying rate closer to $0.311 = 87/280$; Section 5), and the wrong behaviour – incorrect `%.17g` formatting of 1×10^{-7} – is itself the population’s *modal* answer: 26 of the 40 single-agent runs in the RQ1 grid share it. Both aggregation methods therefore do worse than the single agent alone (K 7/40, C 6/40, versus A 14/40), against an oracle ceiling of 37/40 (at least one of $k = 7$ independent attempts passing).

The pre-registered calibration for arm K’s selector (`paper/amendments.md`, `printf` calibration entry) traces this precisely rather than leaving it as a bare number. The 40-attempt candidate pool partitions into exactly two signature classes – 14 passing candidates agreeing with each other on every one of 9650 README-derived execution probes, and 26 failing candidates likewise agreeing with each other on every probe, the two classes disagreeing only on the 24 probes that exercise the one known trap. At $k = 7$, a sampled group ties within whichever cluster is present and MBR-exec’s winner is simply whichever cluster is the local majority in that draw; because the true pass rate is 0.35, the local majority is usually the wrong cluster (two independent closed forms bound the honest range without simulation: hypergeometric, drawing 7 without replacement from the fixed 14-pass/26-fail pool, gives 0.1789; binomial, treating each draw as an independent 0.35 coin flip, gives 0.1998; the realized arm-K rate, $7/40 = 0.175$, sits just below both closed-form figures, close enough that the calibration’s mechanism – not a distinct effect – is the more likely explanation for the observed rate). The three-seat chatroom fails on 34 of 40 runs; the committed artifacts record the oracle’s pass/fail verdict for each chatroom run but not which specific wrong answer a failing run produced, so we cannot cite evidence that all 34 chatroom failures are the same `%.17g` case rather than a mix of independent misreadings. What the pass-rate alone still supports is that arm C’s chatroom process does not correct the single-model bias behind `bench-printf-format`’s failures any better than arm K’s selector does – both land within one failure of each other (6/40 and 7/40) – not a claim about which specific wrong answer each failing run produced.

7.3 Does deliberation add anything beyond sampling?

This room’s brief asked, before any arm-K number existed, whether the chatroom’s specific mechanics – blind openings against anchoring, a mandatory challenge before quorum, quote-verified votes, an independently assigned verifier – contribute anything a plain sampling-and-selection baseline at the same cost does not. K vs. C is not significant on any of the three tasks under Fisher’s exact test with Holm-Bonferroni correction across the pre-registered family of six comparisons (Section 5); the family-pooled comparison on the two `stamp` tasks (K 75/80 vs. C 80/80) is exploratory, outside that pre-registered family, and itself falls short of the conventional 0.05 threshold ($p = 0.059$). Taken alone, that would read as “no distinguishable difference between deliberation and sampling on this family.” The full, pre-registered set of comparisons complicates that reading rather than confirming it.

`paper/prereg-arm-k.md`’s prediction 1 makes three separate claims, and the paper scores all three as written rather than the two `paper/amendments.md` scored when the arm-K result first came in: (i) point forecasts of about 0.99 on `stamp-interpreter` and about 0.90 on `stamp-2`; (ii) K vs. C not significant on either task; (iii) K vs. A *significant* on both tasks. Claim (ii) held on both tasks. Claim (i) held, loosely, for `stamp-2` (forecast 0.90, observed $39/40 = 0.975$) but not for `stamp-interpreter` (forecast 0.99, observed $36/40 = 0.90$), for the reason given in Section 5: the forecast resampled (with replacement) the 40 arm-A answers already collected in the RQ1 grid, and that grid’s pass rate (0.825) itself overstated a freshly measured, larger and independently drawn

single-attempt rate (0.675; Section 5). Claim (iii) is where the two **stamp** programs diverge: K beats A significantly on **stamp-2** (39/40 vs. 28/40, raw $p = 0.0015$, Holm $p = 0.0090$), matching the prediction, but *not* on **stamp-interpreter** (36/40 vs. 33/40, raw $p = 0.518$, Holm $p = 1.000$) – while arm C beats arm A significantly on *both* **stamp** programs (Section 5).

We report that asymmetry as a fact and stop there, because concluding a difference between arm K and arm C from a difference in whether each one’s comparison against arm A reaches significance would be the difference-in-significance fallacy: “K vs. A is significant here and C vs. A is significant there” is not a test of whether K and C differ from each other. The direct test of that question is K vs. C, and it is not significant on **stamp-interpreter** (Section 5). So the asymmetry in claim (iii) is a real, pre-registered pattern worth recording – and **stamp-interpreter**’s own single-agent rate was independently shown to be volatile across samples (0.50, 0.825, and 0.675 at $n = 10$, $n = 40$, and $n = 400$ respectively; Section 5), which is at least as plausible an explanation for a comparison this close to the 0.05 line landing where it did – but it does not, by itself, license a claim that deliberation and sampling differ on this program. We record it because the pre-registration asked for all three of prediction 1’s clauses to be scored, not because it carries evidential weight beyond the direct K-vs-C test already reported.

What the data support, stated at the size the evidence actually is: the chatroom beats the single agent significantly on each **stamp** program individually (Fisher’s exact test, Section 5) and on the two programs pooled as one family ($p < 0.001$); the direct comparison between the chatroom and the oracle-free sampling baseline (K vs. C) is not significant on either program, nor on the family-pooled, exploratory comparison ($p = 0.059$). If the chatroom’s specific consensus mechanics contribute something that independent sampling does not, this suite – two programs in one failure-mechanism family, one task in another – is not powered to detect it, and a reader should not extrapolate a general claim about deliberation, in either direction, from these three tasks alone.

8 Limitations and Future Work

8.1 Research questions not run

This study executed RQ1 (and its arm-K extension) only. RQ2 – the chatroom against a no-chat builder-plus-reviewer pair (protocol §2.1, arm B) – and RQ3 – per-mechanic ablation across the seven hub mechanics listed in protocol §2.3 – are both pre-registered but have no execution behind them in this codebase: arm B has never been run on any task here (**paper/budget.md** §5: “the two cheapest-looking conditions are also the two with no cost evidence at all”), and none of the seven ablations has a real-model run. Both are listed as future work, not reported as negative results.

8.2 Task-suite retirements

bench-long-brief was retired from this suite (**paper/amendments.md**, 2026-09-18): its hand-off/survival scoring is structurally unmeasurable on a single-seat arm A, so it cannot support an accuracy comparison across arms under this study’s method. About 45 candidate task designs were piloted in all, including the three admitted (**paper/amendments.md**: “About 45 candidate designs in all; three are in band”); the roughly 42 that were not admitted span cross-document retrieval, data cleaning, layered rule precedence, tick-scheduler simulation, and several invented-machine languages, and were rejected either at ceiling (3 of 3, or above 0.9 on ten seeds) or as ambiguous. None of that negative-result space is represented in the primary comparison; it is summarized, not reproduced, in **paper/amendments.md**.

8.3 Two families is not a benchmark

The admitted suite spans two failure-mechanism families (scope-capture; exact numeric formatting), not a broad benchmark. A mechanic, or an aggregation strategy, that helps on one family and hurts on the other is exactly the kind of result this suite exists to surface, but three tasks in two families cannot support a general claim about "code tasks" or "aggregation" beyond these two specific mechanisms (Section 6).

8.4 System-level cost and process findings are separate contributions

The token-consumption round (`docs/token-round-0918.md`) and the peer-review round (`docs/review-round-0918.md`) measure this project's own build process, not the arms compared in Section 5; they are reported as system contributions in their own right (Section 3) with their own methodology, and are not folded into RQ1's task-success or cost-per-correct-answer numbers.

8.5 Future work

Completing RQ2 and RQ3; extending the window-control design (Section 5) to `stamp-2` and `bench-printf-format`, which the redesigned, reduced-scope control (Section A) does not cover; a task suite broad enough to support family-level generalization across more than two failure mechanisms; and, if the chatroom's mechanics are ever shown to add value beyond sampling on a broader suite, testing whether a learned or structural minority-override mechanism [8] could recover further value specifically on tasks like `bench-printf-format`, where the correct answer is not the population's plurality and no amount of aggregation-by-agreement can find it.

The mid-study regime shift documented in Section 6 motivates three concrete protocol changes for any future run on this harness, stated in `paper/amendments.md` and repeated here because they bear directly on reproducibility: record output tokens per run as a regime indicator so a shift is visible immediately rather than discovered after the fact; pin a dated model snapshot where the provider allows it, rather than relying on a model alias whose behavior can change mid-session with no corresponding version bump; and never pool runs collected across a detected regime boundary into one estimate, even when, as here, the headline pass-rate numbers turn out not to have been affected.

A Reproducibility Appendix

A.1 Build

Repository `agent-chatroom-mcp`, the commit named in `paper/README.md` (the integration branch this paper was built from; `npm install` is required before any generator script below, since they run under `tsx` against this repository's own dependencies).

```
npm install
npm run build
npm test
```

`npm test` runs `scripts/offline-runner.mjs`; its printed command count drifts over time and must be re-read at the commit actually used, not cited from this text (`paper/system.md` §5).

A.2 RQ1 grid: arms A and C, seeds 101–140

```
node --import tsx scripts/bench-grid.ts \  
  --tasks stamp-interpreter,stamp-2,bench-printf-format \  
  --seeds 101,102,103,104,105,106,107,108,109,110,111,112,113,114,115,116,117,118,119,120, \  
    121,122,123,124,125,126,127,128,129,130,131,132,133,134,135,136,137,138,139,140 \  
  --arms A,C --model sonnet --max-cost-usd 95 \  
  --results-dir bench/results/rq1-suite --port-base 19850
```

(paper/amendments.md, "Grid command and cost estimate for the primary suite"; the seed list is written out in full because macOS's `seq -s, 1 40` appends a trailing comma that `scripts/bench-grid.ts`'s argument parser would otherwise read as a 41st, empty seed. The grid always runs arm C before its paired arm A for a given (task, seed), because arm A's budget is derived from arm C's realized spend.)

A.3 Arm K grid, seeds 101–140

```
node --import tsx scripts/bench-grid.ts \  
  --tasks stamp-interpreter,stamp-2,bench-printf-format \  
  --seeds 101,102,103,104,105,106,107,108,109,110,111,112,113,114,115,116,117,118,119,120, \  
    121,122,123,124,125,126,127,128,129,130,131,132,133,134,135,136,137,138,139,140 \  
  --arms K --k stamp-interpreter=10,stamp-2=8,bench-printf-format=7 \  
  --model sonnet --results-dir bench/results/rq1-arm-k --port-base 19850
```

paired against the arm-C results already written to `bench/results/rq1-suite` (`scripts/bench-grid.ts` refuses an arm-K group whose paired arm-C result does not exist under `--results-dir`). Each group internally invokes `scripts/bench-ak.ts TASK_DIR K SEED --root ... --arm-c-result ...`, running k unmodified arm-A attempts sub-seeded `seed*1000+i` and selecting one by the oracle-free rule for that task's kind (plurality of normalized answers for the two `stamp` tasks; MBR-exec execution agreement, primary, plus signature plurality, exploratory, for `bench-printf-format`; see Section 4).

A.4 Window control

Originally designed as arm C and arm K interleaved per seed on `stamp-interpreter` and `stamp-2`, 20 seeds (201–220), writing to `bench/results/rq1-window-control` (progress log: `logs/rq1-window-control.log`, not committed – `logs/` is gitignored). This run was stopped after 3 runs (\$5.76) when the post-shift cost increase described in Section 6 made the planned 80 runs exceed the control's budget, and was redesigned, entirely inside the post-shift regime, at reduced scope: `stamp-interpreter` only, 12 seeds (201–212), log `logs/rq1-window-control-2.log` (paper/amendments.md, 2026-09-19 thinking-budget entry). Both the original and redesigned runs match `scripts/bench-grid.ts` run the same way as the two grids above, with the corresponding `--tasks`, `--seeds`, and `--results-dir` flags; no committed wrapper script names either exact invocation, so a reader reproducing them should confirm the flags against `scripts/bench-grid.ts`'s own argument parser at the commit named in §A. Its result is reported in Section 5 as complete or pending as of that section's writing, never read from partial output.

A.5 Tables, statistics, figures, and the prose-number audit

```
bash paper/regen.sh
```

`paper/regen.sh` runs every generator listed in `paper/regen-manifest.sh` – `scripts/paper-rq1-table.ts`, `scripts/paper-rq1-armk.ts`, `scripts/paper-rq1-family.ts`

and `scripts/paper-rq1-tex.ts` (results tables and statistics), and `scripts/paper-fig-data.ts`, `scripts/paper-fig-tables.ts` and `scripts/paper-fig.py` (figures) – diffs everything under `bench/results/`, `paper/tables/`, `paper/figures/` and `paper/generated/` against the working tree, and fails if regenerating changes anything `main.tex` already includes. It then runs `scripts/paper-number-audit.ts`, which extracts every decimal, fraction, and p -value written into `paper/sections/*.tex` prose and fails on any that does not appear in a generated table, a committed `bench/results` table, or one of the pre-registration/decision documents (`paper/prereg-arm-k.md`, `paper/amendments.md`, `paper/protocol.md`, `paper/budget.md`, `paper/figures.md`, `docs/token-round-0918.md`, `docs/review-round-0918.md`, `tasks/SUITE.json`) – so no number in the prose can drift from what a generator or a pre-registered constant actually says. Every generator reads only committed `result.json` files under `bench/results/`; none accepts a hand-typed number, and `scripts/rq1-stats.ts` implements the two-proportion z -test, Fisher’s exact test, and Holm-Bonferroni correction with no external statistics dependency.

A.6 Citation check and PDF build

```
paper/check-citations.sh
tectonic paper/main.tex
```

`paper/check-citations.sh` resolves every arXiv id cited in `paper/*.md` and `paper/refs.bib` against `export.arxiv.org` and checks the cited title matches; it does not check numbers, only citations (Section A’s number audit is a separate step, above). Build tooling installed for this room, user-locally only (recorded with these exact commands in `paper/README.md`): `brew install tectonic`; `pip3 install --user --break-system-packages matplotlib` (the `--break-system-packages` flag was required on the build machine used, whose system Python refuses an unqualified `--user` install under PEP 668).

Author and AI-Disclosure Statement

Authors. [Maintainer name/affiliation — to be filled by the maintainer before submission; arXiv’s authorship policy forbids listing any AI agent, model, or seat name as an author]. Corresponding author: [maintainer email].

AI-disclosure statement. This paper describes and evaluates `agent-chatroom-mcp`, a coordination hub whose participants are large-language-model agents (Claude and, in earlier runs, OpenRouter-hosted models); the system under study is itself built from LLM agent output. In addition, large parts of the implementation, the evaluation harness, and drafts of this paper’s text were produced by LLM agents operating under human direction and review, coordinated through the same hub described in the paper (see Section 3 for the architecture; the agents drafting this paper’s sections ran inside an instance of the system they describe). Per arXiv’s generative-AI policy (<https://info.arxiv.org/help/moderation/index.html>), no AI agent is listed as an author; the human author(s) named above take full responsibility for all content, verified every empirical claim and citation before submission, and disclose this use as the paper’s primary methodological instrument, not an incidental aid.

References

- [1] Anshul Bagaria, Sowmya S Sundaram, Gokul S Krishnan, and Balaraman Ravindran. Judging llm-as-a-judge: Concerning rubric artifacts in llm-based automated text generation evaluation, 2026. URL <https://arxiv.org/abs/2609.02942>.
- [2] Niclas Boehmer, Martin Bullinger, and Anna Maria Kerkmann. Causes of stability in dynamic coalition formation, 2022. URL <https://arxiv.org/abs/2211.17169>.
- [3] Martin Bullinger and René Romen. Stability in online coalition formation, 2023. URL <https://arxiv.org/abs/2312.09119>.
- [4] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests, 2022. URL <https://arxiv.org/abs/2207.10397>.
- [5] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance, 2023. URL <https://arxiv.org/abs/2305.05176>.
- [6] Musa Cim, Burak Topcu, Chita Das, and Mahmut Taylan Kandemir. Parallel context compaction for long-horizon llm agent serving, 2026. URL <https://arxiv.org/abs/2605.23296>.
- [7] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. URL <https://arxiv.org/abs/2305.14325>.
- [8] Chuan He, Zebin Chen, Zhengyi Yang, Shaobo Qiao, Mingchen Ju, Jiate Liu, Dong Wen, and Guanfeng Liu. Minority sentinel: When to overturn majority voting in multi-agent llm debates, 2026. URL <https://arxiv.org/abs/2606.29270>.
- [9] Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models?, 2024. URL <https://arxiv.org/abs/2404.06654>.
- [10] Xuanfa Jin, Zhijian Ma, Yongcheng Zeng, Xinyu Cui, Haifeng Zhang, and Jun Wang. Remember and reweight: Enhancing multi-agent debate with experience memory and confidence estimation, 2026. URL <https://arxiv.org/abs/2609.03619>.
- [11] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for ”mind” exploration of large language model society, 2023. URL <https://arxiv.org/abs/2303.17760>.
- [12] Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. More agents is all you need, 2024. URL <https://arxiv.org/abs/2402.05120>.
- [13] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate, 2023. URL <https://arxiv.org/abs/2305.19118>.
- [14] Yasmin Moslem, Magdalena Kacmajor, Vasudevan Nedumpozhimana, Ammar Abbas, Solmaz Panahi, David Lynch, Zhuangzhuang Nie, Alexandros Agapitos, Aleksandar Milenovic,

- Hongmeng Song, Yucheng Shi, Yue Pan, Patricia Buffini, and John D. Kelleher. Cluster, route, escalate: Cascaded framework for cost-aware llm serving, 2026. URL <https://arxiv.org/abs/2606.27457>.
- [15] Tien Anh Nguyen, Khanh-Binh Nguyen, Van Dai Do, Svetha Venkatesh, and Hung Le. Creative generation via multi-agent debate: Does debate suppress diversity?, 2026. URL <https://arxiv.org/abs/2609.00683>.
 - [16] Tuomas Sandholm, Kate Larson, Martin Andersson, Onn Shehory, and Fernando Tohme. Anytime coalition structure generation with worst case guarantees, 1998. URL <https://arxiv.org/abs/cs/9810005>.
 - [17] Freda Shi, Daniel Fried, Marjan Ghazvininejad, Luke Zettlemoyer, and Sida I. Wang. Natural language to code translation with execution, 2022. URL <https://arxiv.org/abs/2204.11454>.
 - [18] Andries Smit, Paul Duckworth, Nathan Grinsztajn, Thomas D. Barrett, and Arnu Pretorius. Should we be going mad? a look at multi-agent debate strategies for llms, 2023. URL <https://arxiv.org/abs/2311.17371>.
 - [19] Minsoo Song, Chanwoo Kim, Sugyeong Eo, and Chanjun Park. Beyond consensus: Downward bias and role asymmetry in multi-agent llm judges for subjective evaluation, 2026. URL <https://arxiv.org/abs/2608.30373>.
 - [20] Jin Su, Runnan Fang, Yeqiu Li, Xiaobin Wang, Shihao Cai, Pengjun Xie, Ningyu Zhang, and Fajie Yuan. U-fold: Dynamic intent-aware context folding for user-centric agents, 2026. URL <https://arxiv.org/abs/2601.18285>.
 - [21] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2022. URL <https://arxiv.org/abs/2203.11171>.
 - [22] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL <https://arxiv.org/abs/2308.08155>.
 - [23] Binwei Yao, Chao Shang, Wanyu Du, Jianfeng He, Ruixue Lian, Yi Zhang, Hang Su, Sandesh Swamy, and Yanjun Qi. Peacemaker or troublemaker: How sycophancy shapes multi-agent debate, 2025. URL <https://arxiv.org/abs/2509.23055>.
 - [24] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL <https://arxiv.org/abs/2306.05685>.